



Reporte De Actividades De Estadía TSU

Robot Diferencial Con Control De
Trayectoria Mediante Perfil
Trapezoidal Y PID

Que Presenta:

Ismael Isaac Abundis Martínez

Asesor Institucional:

Dr. Manuel Benjamín Ortiz Moctezuma

Nombre De Unidad Económica:

Centro De Investigación Y De Estudios
Avanzados Del Instituto Politécnico Nacional

Cd. Victoria, Tamaulipas a 14 De Agosto Del 2026

(Parte 1 de 2)

Formato de autorización para exposición de Estadía de
Técnico Superior Universitario

Cd. Victoria, Tamaulipas a 14 de agosto del 2026

Ismael Isaac Abundis Martínez

#2430256

Ingeniería mecatrónica

Por medio del presente, se le informa que tras realizar su Estadía en la empresa Centro De Investigación Y De Estudios Avanzados Del Instituto Politécnico Nacional llevando a cabo el proyecto Robot Diferencial Con Control De Trayectoria Mediante Perfil Trapezoidal Y PID durante el periodo comprendido del 4 de mayo al 14 de agosto logrando una ponderación de 57.6 (60% posibles) para el criterio de reporte; por lo tanto, se otorga la oportunidad de exponer el trabajo realizado por medio de un informe ejecutivo programado para el día 17 del mes de agosto del año en curso en un horario de 11:00 a 11:30 am en modalidad presencial en el aula B-205.

Sin otro particular, se le recomienda estar al disponible para iniciar su exposición 10 minutos antes de la indicación oficial.

Atentamente
Manuel Benjamin Ortiz Maldonado
Ben

Asesor institucional

(Parte 2 de 2)

Control sumativo de Estadía para Técnico Superior Universitario

Criterio	Valor / Ponderación	Calificación
Reporte	60%	57.6
Exposición	40%	40
Calificación final	97.6	

Nota: para completar este documento deberá sustentar su decisión con base en las rúbricas oficiales de la institución diseñadas para cada evaluación.

Datos de la evaluación sumativa

17 18 12026

Manuel Benjamin Ortiz Maldonado
Ben
Asesor institucional

Resumen

Este trabajo de estadías TSU, desarrollado en el Laboratorio de Robótica de CINVESTAV Unidad Tamaulipas para obtener el título de Técnico Superior Universitario en Mecatrónica, presenta el diseño, construcción e instrumentación de un robot móvil de tracción diferencial con seguimiento de trayectoria mediante perfil trapezoidal de velocidad y control de doble retroalimentación. La metodología se estructuró en cuatro etapas: acondicionamiento del banco de pruebas con motores DC con *encoders*; desarrollo de controladores P de posición, PID de velocidad con filtro de media móvil exponencial y generadores de perfil trapezoidal; deducción de la cinemática directa e inversa junto con telemetría web asíncrona mediante el protocolo *WebSockets*; y diseño CAD e impresión 3D del chasis circular en filamento PETG. Los resultados demostraron un seguimiento preciso y suave de trayectorias, obteniendo un desfase lineal de 2 cm por metro en desplazamientos físicos. Se concluyó que la plataforma mecatrónica ofrece una navegación móvil diferencial precisa, estable y altamente eficiente.

Palabras clave: Robot diferencial, perfil trapezoidal, control PID, doble retroalimentación, ESP32, WebSockets.

Abstract

This TSU internship work, carried out in the Robotics Laboratory of CINVESTAV Tamaulipas Unit to obtain the title of University Higher Technician in Mechatronics, presents the design, construction, and instrumentation of a differential-drive mobile robot with trajectory tracking using a trapezoidal speed profile and dual feedback control. The methodology was structured in four stages: conditioning the test bench with DC motors with encoders; development of P position controllers, PID speed controllers with exponential moving average filter, and trapezoidal profile generators; deduction of direct and inverse kinematics along with asynchronous web telemetry using the WebSockets protocol; and CAD design and 3D printing of the circular chassis in PETG filament. The results showed precise and smooth trajectory tracking, achieving a linear lag of 2 cm per meter in physical movements. It was concluded that the mechatronic platform provides accurate, stable, and highly efficient differential mobile navigation.

Keywords: Differential robot, trapezoidal profile, PID control, dual feedback, ESP32, WebSockets.

Contenido temático.

Introducción	5
Descripción de la Unidad Económica	6
Objetivos operativos.....	7
Metodología	9
Primera etapa:.....	9
Segunda etapa.....	14
Tercera Etapa	30
Cuarta Etapa.....	35
Resultados	39
Etapa 1:	39
Acondicionamiento del hardware y software base.....	39
Etapa 2:	40
Control de posición con controlador proporcional	40
Control de velocidad con controlador PID	43
Perfil trapezoidal con controlador de velocidad PID.....	47
Perfil trapezoidal con controlador de posición PD con doble retroalimentación. .	50
Sincronización de perfiles.....	53
Etapa final:	56
Armado del robot diferencial	56
Robot diferencial con perfil trapezoidal de velocidad y telemetría.....	58
Pruebas del desplazamiento lineal del robot diferencial con perfil trapezodial de velocidad.....	64
Conclusiones.	67
Referencias.....	68

Introducción

El presente trabajo de estadía profesional se llevó a cabo en el Laboratorio de Robótica del Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional (CINVESTAV) Unidad Tamaulipas, ubicado en el Parque Científico y Tecnológico de Tamaulipas en Ciudad Victoria. Este centro de excelencia científica se distingue por su investigación de frontera en mecatrónica, control automático y sistemas autónomos. Dentro de esta institución, las actividades se desarrollaron en el área de instrumentación robótica, asumiendo la responsabilidad técnica de diseñar, construir y validar un prototipo funcional de robot móvil de tracción diferencial con capacidad de procesamiento embebido en tiempo real y supervisión inalámbrica.

En la navegación de plataformas móviles diferenciales, la aplicación directa de consignas de movimiento sin suavizado representa un problema técnico crítico, ya que demanda aceleraciones infinitas teóricas que derivan en picos de corriente destructivos para los controladores de potencia. Esta brusquedad provoca el deslizamiento inevitable de las ruedas sobre la superficie de rodamiento, corrompiendo la precisión de la odometría y distorsionando la estimación de posición del vehículo en el espacio. Asimismo, la ausencia de una regulación coordinada de velocidad y posición limita la fluidez de las trayectorias y propicia errores residuales en el posicionamiento final.

Para resolver estas deficiencias, se desarrolló una metodología mecatrónica integral que inició con la caracterización e instrumentación del hardware en un microcontrolador ESP32 programado en PlatformIO/VS Code. Se sintetizaron generadores analíticos de perfiles trapezoidales de velocidad, un esquema de control PD de doble retroalimentación y filtros exponenciales EMA para la atenuación del ruido de cuantización. Complementariamente, se dedujo la cinemática de la plataforma, se implementó una interfaz de telemetría web asíncrona mediante WebSockets y se diseñó un chasis circular manufacturado en PETG. Los resultados mostraron transiciones de movimiento altamente fluidas, eliminación total de picos de corriente y una excelente repetibilidad con un desfase lineal de solo 2 cm por metro recorrido.

Descripción de la Unidad Económica

El Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional (CINVESTAV) Unidad Tamaulipas es una institución pública dedicada principalmente a la investigación científica, el desarrollo tecnológico y la educación de posgrado a nivel de maestría y doctorado. Su giro principal se enfoca en el desarrollo de soluciones computacionales, la ingeniería de software y el diseño de sistemas aplicados para resolver problemáticas tecnológicas y de la industria. Las instalaciones de esta unidad económica se encuentran ubicadas en el Kilómetro 5.5 de la Carretera Victoria-Soto La Marina, dentro del Parque Científico y Tecnológico de Tamaulipas, en Ciudad Victoria, Tamaulipas. Este complejo proporciona un entorno adecuado y profesional para la vinculación entre el sector académico y el desarrollo de proyectos tecnológicos.

Dentro de esta institución, las actividades de la estadía profesional se desarrollan específicamente en el Laboratorio de Robótica. Este departamento está orientado al diseño, programación e integración de sistemas mecatrónicos, con un enfoque particular en la navegación autónoma, el control de sistemas dinámicos y la implementación de algoritmos en hardware físico. La misión de este laboratorio es generar investigación aplicada de calidad y formar especialistas con la capacidad técnica necesaria para abordar y resolver problemas en el área de la automatización. Por su parte, la visión del departamento es consolidarse y mantenerse como un espacio de referencia en la robótica móvil, impulsando constantemente la actualización de sus procesos y proyectos de investigación.

Para llevar a cabo estas actividades, el Laboratorio de Robótica cuenta con una infraestructura sumamente práctica, funcional y en constante uso, la cual se caracteriza por mantenerse al día con las necesidades del trabajo diario, evitando por completo la acumulación de equipo obsoleto o fuera de servicio. En el área de manufactura y diseño mecánico, el departamento dispone de varias impresoras 3D y un suministro constante de filamentos poliméricos, lo que permite el prototipado rápido de estructuras, engranes y soportes a la medida para los proyectos.

Adicionalmente, el laboratorio cuenta con una variedad de plataformas físicas de evaluación. Entre estas destacan diversos vehículos robóticos terrestres y vehículos aéreos no tripulados (drones), los cuales funcionan como bases experimentales para que los estudiantes e investigadores puedan probar y validar los lazos de control y la electrónica desarrollada. Finalmente, el espacio de trabajo físico fomenta la colaboración y el análisis técnico, por lo que está equipado con áreas provistas de pizarrones amplios y proyectores que facilitan el desarrollo de modelos matemáticos, la planeación de la lógica de programación y la discusión de resultados. En conjunto, estos recursos proveen el entorno técnico ideal para el desarrollo y culminación del proyecto de control diferencial asignado.

Objetivos operativos

Para poder desarrollar este proyecto de estadía de manera estructurada y eficiente, se establecieron principalmente los siguientes objetivos operativos. Estos fueron pensados estratégicamente en conjunto con el asesor institucional para lograr un desarrollo orgánico e incremental, permitiendo avanzar paso a paso desde el entendimiento de la electrónica básica hasta la integración de algoritmos de control complejos.

En este sentido, estos objetivos definieron el rumbo exacto a tomar dentro del laboratorio. Al detallar las acciones a realizar y las herramientas necesarias para ejecutarlas, funcionaron como una guía clara para las actividades diarias y establecieron las bases directas que conforman las etapas de la metodología. Para alcanzar la meta de integrar el robot diferencial, se plantearon los siguientes puntos:

- Acondicionar el hardware y software base del sistema de actuación y medición del motor DC, mediante la configuración de señales *PWM* para el puente H, el uso del *ADC* para lecturas analógicas, la implementación de interrupciones externas (ISR) para pruebas iniciales, la decodificación de *encoder* en cuadratura utilizando el periférico *PCNT* del ESP32, y la familiarización con herramientas de visualización como el *Serial Plotter*, con el fin de dejar todo listo para la implementación de estrategias de control.
- Implementar y sintonizar controladores P, PI y PID para la regulación de velocidad y posición, cuyas consignas serán asignadas correspondientemente por un perfil trapezoidal de velocidad y un perfil parabólico por tramos). Posteriormente, integrar una arquitectura de control de posición con doble retroalimentación (posición con

término derivativo de velocidad) para seguir dicha trayectoria, y finalmente sincronizar dos motores bajo este esquema, asegurando que ambos actuadores recorran distancias diferentes (para giros o desplazamientos diferenciales) en un mismo intervalo de tiempo, manteniendo aceleraciones y velocidades de cruce coordinadas.

- Analizar el modelo cinemático del robot de tracción diferencial, abordando tanto el modelo cinemático directo (velocidades de las ruedas a velocidad y orientación del robot) como el inverso (velocidad y orientación deseada a velocidades de ruedas), e implementar un control cinemático que permita traducir consignas de movimiento global en referencias individuales para cada actuador.
- Construir el prototipo funcional del robot móvil, mediante el diseño mecánico asistido por computadora (CAD), la impresión 3D del chasis, el ensamble e integración de la electrónica de potencia y control (drivers, microcontrolador, batería), y la instrumentación completa para la fase de pruebas de los perfiles trapezoidales de velocidad, validando desplazamientos.

Metodología

El desarrollo del proyecto se dividió en cuatro etapas, cada una correspondiente directamente a un objetivo operativo. A continuación, se describe cronológicamente la primera de ellas.

Primera etapa:

Esta etapa tuvo como finalidad habilitar físicamente el banco de pruebas para poder generar movimiento en el motor y leer su posición real a través del *encoder*. Para ello, se partió del módulo cuadrado impreso en 3D que ya contenía el motorreductor JGA25 (relación 46:1, *encoder* de 11 pulsos por revolución con cuadratura completa) y el puente H IBT_2 basado en BTS7960. El trabajo comenzó con una fase de documentación técnica intensiva, considerada un paso fundamental para evitar daños a los componentes y para conocer las capacidades y limitaciones del microcontrolador ESP32, que sería el cerebro del sistema.

El primer paso fue la lectura detallada de la hoja de datos del ESP32, enfocándose en los módulos que se utilizarían a lo largo del proyecto: el generador de *PWM* conocido también como LED *PWM* Controller (LEDC), el convertidor analógico-digital (ADC), los puertos *GPIO* y, de manera especial, el contador de pulsos (*PCNT*) para decodificar las señales en cuadratura del *encoder*. Durante esta lectura, se comprendió que el ESP32 dispone de 16 canales independientes de *PWM* pero únicamente 4 temporizadores para gobernarlos, lo que implicaría planificar cuidadosamente la asignación de estos recursos cuando más adelante se controlarán dos motores simultáneamente.

Generación de señal PWM.

Se identificó la relación entre la resolución en bits y la frecuencia máxima que puede alcanzar cada canal, dato crítico para elegir la configuración más adecuada. De acuerdo con las especificaciones del fabricante [1], para una resolución de 12 bits se puede lograr hasta 15 kHz, mientras que reduciendo a 11 bits se alcanzan 20 kHz, rozando los límites superiores del hardware. Se decidió optar por 11 bits y 20 kHz como punto de partida, priorizando una frecuencia de conmutación suficientemente alta para reducir el ruido audible y mantener un control fino de velocidad, sin exceder la capacidad de respuesta del puente H IBT_2, cuyos datos técnicos indican un rango de operación entre 1 kHz y 25 kHz.

Tabla 1. Frecuencias y resoluciones típicas [1].

<i>Fuente de reloj (LEDC_CLKx)</i>	<i>Frecuencia PWM</i>	<i>Resolución Máxima (bits)</i>	<i>Resolución Mínima (bits)</i>
<i>APB_CLK (80 MHz)</i>	1 kHz	16	7
<i>APB_CLK (80 MHz)</i>	5 kHz	13	4
<i>APB_CLK (80 MHz)</i>	10 kHz	12	3
<i>RC_FAST_CLK (8 MHz)</i>	1 kHz	12	3
<i>RC_FAST_CLK (8 MHz)</i>	2 kHz	11	2
<i>REF_TICK (1 MHz)</i>	1 kHz	9	1

En la Tabla 1 se resumen las combinaciones típicas de frecuencia de conmutación y resolución para las distintas fuentes de reloj del periférico LEDC [1]

La frecuencia de salida esta dada por la siguiente ecuación [1]:

$$f_{OUT} = \frac{f_{LEDC_CLK}}{LEDC_CLK_DIV \cdot 2^{DUTY_RESOLUTION}}$$

Donde f_{LEDC_CLK} es la frecuencia de reloj del timer en uso, por defecto se usa el APB_CLK de 80MHz, $LEDC_CLK_DIV$ es un factor divisor que regula los pulsos del *timer* que le llegan al canal *PWM*, $2^{Duty_resolution}$ se refiere a la resolución del ciclo de trabajo. En la hoja técnica se especifica que el divisor de reloj tiene que ser igual o mayor que uno y menor a 1023, en base a esto podemos calcular una frecuencia y resolución dada sin pasar de las limitaciones físicas del hardware.

$$LEDC_CLK_DIV = \frac{f_{LEDC_CLK}}{f_{OUT} \cdot 2^{DUTY_RESOLUTION}}$$

Entonces con una resolución de 11 bits a 20 KHz:

$$LEDC_CLK_DIV = \frac{80 \text{ MHZ}}{20 \text{ Khz} \cdot 2^{11}}$$

$$LEDC_CLK_DIV = 1.95$$

Dado que 1.95 se encuentra dentro de este rango permitido por la hoja técnica, se validó la configuración como viable, evitando así forzar el módulo a un fallo de hardware por divisores menores a 1.

Implementación del puente H y determinación del *PWM* mínimo para vencer la inercia.

Una vez configurados los parámetros de frecuencia y resolución del PWM en el ESP32 (20 kHz, 11 bits), se procedió a integrar el puente H IBT_2 como etapa de potencia entre el microcontrolador y el motor DC. El puente H recibe la señal *PWM* proveniente del pin correspondiente del ESP32, así como una señal digital de dirección, y entrega a los bornes del motor un voltaje modulado en ancho de pulso con la capacidad de corriente necesaria, la cual según los fabricantes [2] es de hasta 43 A, muy por encima de los 2 A que consume el motor JGA25. Para validar el correcto funcionamiento del conjunto y caracterizar el comportamiento del motor ante diferentes ciclos de trabajo, se diseñó una prueba sencilla pero informativa: determinar el valor mínimo de *PWM* que logra vencer la fricción estática del sistema (incluyendo el reductor de 46:1 y el propio rotor) y poner el motor en movimiento al aire libre sin carga. Para ello, se escribió un código de prueba que lee el valor de un potenciómetro de 50 kΩ conectado a una entrada *ADC* del ESP32. El *ADC* del ESP32 tiene una resolución nativa de 12 bits, por lo que entrega valores enteros de 0 a 4095 que representan el voltaje de 0 a 3.3 V aplicado al potenciómetro. Sin embargo, la señal *PWM* que controla la velocidad del motor tiene una resolución de 11 bits (valores de ciclo de trabajo de 0 a 2047). Por tanto, dentro del programa se implementó una operación de reescalamiento: el valor leído por el *ADC* (0-4095) se dividió entre 2 (o, más exactamente, se desplazó un bit a la derecha) para obtener un rango de 0 a 2047, compatible con el *PWM*. Este mapeo lineal asegura que, al girar el potenciómetro, el ciclo de trabajo varíe de forma proporcional y continua.

Con el potenciómetro llevado a su valor mínimo, se comenzó a incrementar muy lentamente la referencia mientras se observaba visualmente el eje del motor. Se prestó especial atención al comportamiento del puente H, ya que este componente es el encargado de traducir la señal de baja corriente del ESP32 en un voltaje conmutado de 12 V capaz de suministrar la corriente de arranque del motor. Se comprobó que, para valores muy bajos de *PWM*, el motor permanecía completamente detenido, emitiendo únicamente un zumbido de baja frecuencia producto de la corriente que circula por las bobinas sin lograr generar suficiente par. Al superar un cierto umbral, el rotor comenzó a girar de forma sostenida. Utilizando el monitor serie del entorno de desarrollo, se registró el valor exacto del ciclo de trabajo en el instante en que se producía el movimiento: 350 unidades sobre 2047 posibles. Este valor equivale aproximadamente a un 17% del ciclo de trabajo activo ($350 / 2047 \approx 0.171$). Por debajo de este umbral, el motor solo vibraba sin girar, evidenciando la existencia de una zona muerta atribuible a la fricción estática del reductor y a la inercia del conjunto, cabe mencionar que al montar los motores en un robot será necesario encontrar el umbral correspondiente a la carga de toda la unidad.

Este resultado resultó fundamental para las etapas posteriores, ya que cualquier lazo de control PID que ordene un ciclo de trabajo inferior a 350 no logrará efectivamente mover el motor, provocando un error permanente que el integrador podría intentar acumular sin éxito. Por ello, el valor de 350 se estableció como el mínimo práctico a entregar cuando se requiera movimiento, y se incorporó como un límite inferior en la lógica del controlador para evitar saturación innecesaria.

Lectura del *encoder* incremental.

Una vez validado el funcionamiento del *PWM* y caracterizado el umbral mínimo de arranque del motor, el siguiente paso para cerrar la primera etapa fue implementar la lectura del *encoder*. El motor JGA25 cuenta con un *encoder* incremental de 11 pulsos por revolución, con salida en cuadratura. Antes de escribir código, investigué acerca de los *encoders* incrementales y se definen [3] como dispositivos electromecánicos los cuales generan pulsos a medida que el rotor de un motor gira, a diferencia de los absolutos estos requieren de una referencia inicial para determinar la posición actual. Los pulsos se conforman por dos señales desfasadas 90 grados que permiten determinar tanto la velocidad como la dirección de giro, mientras la cuadratura media solo detecta flancos de una de las señales, la cuadratura completa aprovecha los flancos de subida y bajada de ambas, multiplicando la resolución efectiva por cuatro [4]. Así, partiendo de los 11 pulsos originales y considerando la relación de transmisión de 46:1 del reductor, se obtienen 506 pulsos por vuelta del eje de salida; al aplicar cuadratura completa, esta cifra se eleva a 2024 pulsos por revolución (PPR), una resolución más que suficiente para un control fino.

Mi primer enfoque fue leer el *encoder* por software, usando interrupciones externas en los pines del ESP32. Hice varias prácticas para familiarizarme con el manejo de interrupciones, y en ese proceso aprendí que cualquier variable compartida entre la rutina de interrupción y el programa principal debe declararse como volátil, ya que esas rutinas se ejecutan en RAM y el compilador podría optimizar su acceso de manera incorrecta si no se le indica que su valor puede cambiar en cualquier momento. Programé la lógica de detección de dirección de la siguiente manera: cuando me apoyaba en el canal A, en cada cambio de flanco de este canal, revisaba el estado de ambos canales; si eran iguales, sumaba un pulso, y si eran diferentes, restaba. Cuando me apoyaba en el canal B, en cada cambio de flanco de este, también revisaba el estado de ambos; pero aquí la regla era la inversa: si los dos estaban iguales, restaba, y si eran diferentes, sumaba. Esta lógica funcionó bien, y de hecho no tengo algo lo suficientemente rápido para probar que diese problemas; en general se comportaba de forma correcta. Sin embargo, sabía que el ESP32 tiene un periférico de hardware dedicado para contar pulsos en cuadratura (PCNT), que además libera al procesador de atender interrupciones constantemente. Por eso decidí explorar esa vía, no porque las interrupciones fallaran, sino por una cuestión de buena práctica y eficiencia.

Al investigar el PCNT, encontré dos librerías. La primera, más sencilla, es ESP32Encoder, muy común en el ecosistema Arduino. Con ella, bastaba con definir los pines y especificar si quería cuadratura media o completa; el resto lo resolvía internamente. Un detalle importante es que maneja automáticamente el desbordamiento del contador interno (que es de 32 bits), guardando el valor en una variable de mayor capacidad para no perder la cuenta total. Así, solo tenía que llamar a `encoder.getCount()` cada vez que necesitaba la posición actual. La segunda opción era trabajar directamente con la librería oficial `pcnt.h` de Espressif. Esta me daba un control mucho más fino: podía configurar filtros antirrebote, ajustar umbrales de comparación y personalizar casi cada aspecto del módulo. Pero también implicaba programar desde cero la lógica para que el contador se comportara como un lector de *encoder* en cuadratura completa, lo que requería más líneas de código y más tiempo de depuración.

Después de probar ambas opciones en el banco de pruebas, opté por la librería ESP32Encoder por su practicidad y su buen desempeño. Verifiqué que, tal como esperaba, al configurar cuadratura completa el conteo se multiplicaba por cuatro, alcanzando la resolución teórica de 2048 PPR. Con esto, ya tenía todos los elementos necesarios para cerrar el lazo: por un lado, la generación de velocidad mediante *PWM* con el umbral mínimo de 350 caracterizado; por otro, la medición fiable de posición y velocidad real a través del *encoder*. De esta manera, la primera etapa del proyecto —el acondicionamiento del hardware de actuación y medición— quedó concluida, dejando

el banco de pruebas listo para abordar la siguiente fase: la implementación del control PID en lazo cerrado.

Segunda etapa.

Una vez concluida la primera etapa, el banco de pruebas ya era capaz de leer la posición real del motor a través del *encoder* y de aplicar una velocidad mediante *PWM*. El siguiente objetivo operativo fue implementar estrategias de control que permitieran mover el motor de forma precisa y suave. Esta etapa se dividió en tres pasos: primero un control de posición con un controlador P, luego la incorporación de un generador de trayectorias con perfil trapezoidal, y finalmente la integración de un control en cascada de posición sobre velocidad. A continuación, se describe el primero de ellos.

Control de posición con un controlador proporcional.

Mi doctor me planteó un reto inicial muy claro: lograr que el motor se moviera a una posición angular determinada, sin importar el tiempo ni la velocidad que tomara, pero con dos condiciones indispensables: que alcanzara efectivamente esa posición y que una vez ahí la mantuviera, corrigiendo cualquier desviación por perturbaciones externas. Tenía el conocimiento que esto implicaba un lazo de control cerrado utilizando la lectura del *encoder* como retroalimentación y el *PWM* como señal de actuación. Antes de escribir cualquier código, me dediqué a estudiar los fundamentos del control automático.

En [5] se explica la nomenclatura básica de los controladores introduciendo el término referencia o punto de consigna como el estado deseado del sistema, y el término variable de salida o de proceso para referirse al estado medido del sistema.

Tabla 2. Convenciones comunes de denominación de variables [5].

VARIABLE	DESCRIPCIÓN
$r(t)$	Punto de consigna, referencia
$e(t)$	Error
$u(t)$	Esfuerzo de control
$y(t)$	Salida, variable de proceso

Calculando el error como:

$$e(t) = r(t) - y(t)$$

El término proporcional intenta reducir el error de posición a cero contribuyendo a la señal de control proporcionalmente al error de posición actual.

$$u(t) = K_p e(t)$$

En el caso de un motor, esto se traduce en que, si el error de posición es grande, se entrega un ciclo de trabajo más alto para que el motor gire con mayor velocidad, reduciendo el error rápidamente; a medida que el error disminuye, también disminuye el *PWM*, evitando que el motor sobrepase la consigna. La constante proporcional K_p determina qué tan agresiva es esa respuesta. Valores altos hacen que el motor reaccione fuerte, pero pueden generar inestabilidad u oscilaciones, mientras que valores bajos producen movimientos lentos y posiblemente no se alcance nunca la posición exacta por falta de fuerza.

Llevar esto a la práctica requirió resolver varios detalles de implementación. Decidí que el cálculo del control debía realizarse de manera periódica y no dentro del bucle principal, para garantizar tiempos de muestreo constantes. Para ello configuré un temporizador interno del ESP32 que genera una interrupción cada 20 milisegundos. Dentro de la rutina de interrupción correspondiente (una ISR interna), coloqué toda la lógica del controlador. Dentro de esa ISR, primero leía la posición actual del *encoder* usando la función `encoder.getCount()` de la librería que había seleccionado en la etapa anterior. Luego calculaba el error como la resta entre la consigna de posición y la posición actual. A continuación, multiplicaba ese error por la constante K_p para obtener la salida del controlador. Esta salida era un número que podía ser positivo o negativo, donde el signo indicaba la dirección de giro necesaria para reducir el error. Después venían dos decisiones críticas: la saturación y el sentido de giro.

Para la saturación, definí un valor mínimo y máximo de *PWM* que el motor puede recibir. El valor mínimo lo tomé de la caracterización realizada en la primera etapa,

donde determiné que el motor empezaba a girar con un ciclo de trabajo de 350 (sobre un máximo de 2047). Cualquier salida por debajo de ese umbral no produciría movimiento y el controlador quedaría estancado. El valor máximo lo fijé en 2047, que corresponde al 100% del ciclo de trabajo. Así, después de calcular la salida del controlador, si el valor absoluto era menor a 350, lo forzaba a 350 para asegurar que el motor realmente girara; si superaba 2047, lo limitaba a 2047. El sentido de giro se determinó a partir del signo de la salida del controlador. Si la salida era positiva, eso significaba que el error era positivo, es decir, la posición actual era menor que la consigna y el motor debía girar hacia adelante. Si era negativa, debía girar en reversa. Para controlar la dirección usando el puente H IBT_2, no se emplea un pin de dirección como en otros puentes. En este caso, el sentido se define enviando la señal *PWM* al pin RPWM para girar hacia adelante o al pin LPWM para girar en reversa, mientras el otro pin se mantiene en nivel bajo. Por lo tanto, dentro de la ISR, si la salida del controlador era positiva, escribía el valor absoluto ya saturado en el canal asociado al RPWM y ponía el LPWM en cero; si era negativa, hacía lo contrario. De esta forma, el valor absoluto de la salida determinaba la velocidad a través de la función *ledcWrite* y el signo elegía qué pin del puente H recibía la señal.

Ogata [6] dice que representar un sistema de control dinámico a través de un diagrama de bloques representa una ventaja ya que permite ver con facilidad la contribución de cada componente en el desempeño del sistema. A continuación, el diagrama de bloques de mi sistema:

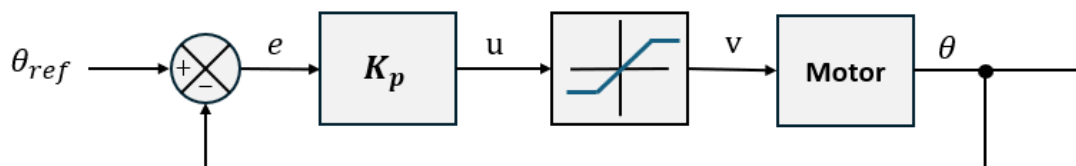


Figura 1 – Diagrama de bloques conceptual del sistema de control proporcional de posición de un motor DC.

La figura 1.0 ilustra el diagrama de bloques del sistema de control en lazo cerrado. En este esquema, el punto de suma compara la posición de referencia θ_{ref} con la posición real θ para obtener el error e . Dicho error entra al controlador de ganancia proporcional K_p , el cual genera la señal de control u . Posteriormente, el bloque de saturación recorta esta señal al límite de voltaje operativo v soportado por el hardware y el código. Finalmente, la salida del motor θ se toma en el punto de ramificación para retroalimentarse directamente al punto de suma.

Con esta implementación, realicé varias pruebas asignando diferentes consignas de posición. Observé que el motor se movía hacia la posición deseada y, una vez ahí, se mantenía corrigiendo pequeñas desviaciones. El comportamiento dependía fuertemente del valor de K_p que eligiera. Probé valores bajos y noté que el motor llegaba muy lento y nunca terminaba de alcanzar la posición exacta, quedándose a unos pulsos de distancia por falta de fuerza suficiente para vencer la fricción estática. Con valores altos, el motor se movía rápido, pero oscilaba varias veces alrededor de la consigna antes de estabilizarse. Finalmente seleccioné una K_p intermedia que daba un equilibrio aceptable entre rapidez y estabilidad. El control proporcional por sí solo, como era de esperarse, presentaba un error en estado estacionario porque al llegar cerca de la consigna el error se hacía pequeño y el PWM también, insuficiente para vencer la fricción. Eso ya lo tenía contemplado y sería resuelto en el siguiente paso con la acción integral. Durante este proceso, utilicé el Serial Plotter para graficar en tiempo real la consigna, la posición medida y la salida del controlador. Esto me permitió visualizar el error y ajustar la constante de forma iterativa. Al finalizar este paso, ya tenía un control de posición funcional. El motor era capaz de ir a cualquier posición indicada y mantenerla, aunque con un pequeño error residual.

Control de velocidad con controlador PI y PID

Una vez que el control de posición proporcional funcionaba correctamente, el siguiente reto fue controlar la velocidad del motor. Para ello, lo primero que necesitaba era calcular la velocidad real a partir de las lecturas del *encoder*. El método fue el siguiente: dentro de la misma interrupción por temporizador que ejecutaba el control cada 20 milisegundos, almacenaba el valor actual del contador de pulsos del *encoder* en una variable. En la siguiente iteración, calculaba la diferencia entre el pulso actual y el pulso anterior. Esta diferencia representaba el cambio de posición ocurrido durante los últimos 20 milisegundos. Luego dividía esa diferencia entre el período de muestreo, que era 0.02 segundos, obteniendo así la velocidad en pulsos por segundo. Una vez hecho el cálculo, actualizaba la variable del pulso anterior igualándola al valor actual para la próxima iteración.

Con la velocidad real ya disponible, escribí un controlador proporcional puro para regular la velocidad. El setpoint se daba en pulsos por segundo y podía ser positivo o negativo según la dirección deseada. Al probar este controlador, me encontré con un comportamiento que mi asesor quería que observara por mí mismo: el motor nunca alcanzaba la consigna de velocidad. La razón es que, en un control puramente

proporcional, la salida es K_p por el error. Si el error es grande, el PWM es alto y el motor acelera. A medida que se acerca a la velocidad deseada, el error disminuye y el PWM también baja. Para mantener una velocidad alta se necesita un PWM sostenido, pero justo cuando el error se vuelve pequeño, el PWM se reduce tanto que el motor termina yéndose a una velocidad menor a la consigna. En otras palabras, el término proporcional por sí solo no puede compensar la carga que exige mantener una velocidad constante, porque al disminuir el error disminuye la fuerza aplicada. Este comportamiento era exactamente la lección que el asesor quería que identificara.

La solución clásica es agregar un término integral. El integral acumula el error a lo largo del tiempo. Así, aunque el error se vaya haciendo pequeño, la suma histórica de esos errores sigue creciendo y aporta una acción de control adicional.

La salida de control PI se define como [5]:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau$$

De esta manera cuando el termino proporcional se vuelva cero, el termino integral ya habrá acumulado justo el valor necesario para mantener el *PWM* que sostiene la velocidad deseada. Implementar esto fue sencillo sobre la base que ya tenía: dentro de la misma ISR, después de calcular el error actual, lo sumaba a una variable acumuladora. Luego multiplicaba el error por K_p y el acumulado por K_i , y sumaba ambos para obtener la salida del controlador. El resto de la lógica de saturación y sentido de giro se mantuvo igual.

Sin embargo, el término integral trae consigo problemas conocidos. Uno de ellos es la saturación del acumulador, también llamado efecto *windup*. Si por alguna razón el motor se bloquea o no puede seguir el *setpoint*, el error se mantiene grande durante mucho tiempo y el acumulador crece sin límite. Cuando el bloqueo desaparece, ese acumulador enorme produce una salida desmedida que hace que el motor se dispare. Para evitarlo, implementé una saturación del acumulado: definí un valor máximo y mínimo que la variable integral podía alcanzar, de modo que nunca creciera más allá de esos límites. Con esto limitaba la acción integral a un rango seguro.

Otro fenómeno que observé está relacionado con el sentido de giro. En el control de velocidad, el *setpoint* lleva un signo que indica la dirección deseada. Mientras el motor gira en esa dirección, el control trabaja normalmente. Pero si la velocidad real sobrepasa el *setpoint*, el error cambia de signo, lo que provoca que la salida del controlador también cambie de signo. Eso hace que el puente H invierta la corriente en el motor, generando un par de frenado. El motor sigue girando físicamente en el mismo sentido, pero ahora el control está aplicando una fuerza contraria para reducir la velocidad. Esta dinámica es correcta y deseable para regular la velocidad sin necesidad de frenos mecánicos. Finalmente, añadí una medida adicional para mejorar la respuesta ante paradas bruscas. Implementé una histéresis que vacía el acumulador integral cuando el *setpoint* es superado por un cierto umbral. ¿Por qué? Imaginemos que se ordena detener el motor, es decir, *setpoint* cero. Si antes el motor iba a alta velocidad, el integral pudo haberse acumulado a un valor grande. Al poner *setpoint* cero, el error es grande y negativo, y el integral empieza a descargarse lentamente. Durante ese tiempo, el motor podría seguir moviéndose o incluso invertir su giro de forma no deseada. Para acelerar la recuperación, definí que, si la velocidad real supera el *setpoint* en más de un valor umbral, la variable acumuladora se reinicia a cero. De esta forma, el controlador vuelve a depender principalmente del término proporcional para regular la situación transitoria, evitando que un integral lleno dispare el sistema al liberarse. El sistema puede ser representado de la siguiente manera:

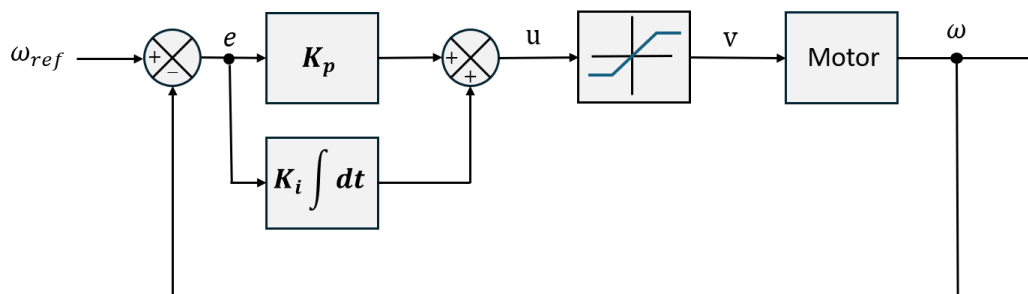


Figura 1.1 – Diagrama de bloques del controlador PI de velocidad de un motor DC.

Con todas estas implementaciones, obtuve un controlador de velocidad PI funcional, sin embargo, era evidente que si quería que el sistema tuviera una corrección rápida se presentaban oscilaciones, el siguiente paso era añadir el termino derivativo. El término derivativo responde a la tasa de cambio del error. Es decir, predice o anticipa cómo variará el error en el futuro inmediato. Su función principal es amortiguar las oscilaciones y aumentar la estabilidad, sirviendo como freno ante cambios bruscos o sobre impulsos [7].

La salida de control con los tres términos [7]:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de}{dt}$$

Añadiendo el termino derivativo se suavizaron las oscilaciones, pero es necesario considerar que este puede introducir ruido si no se sintoniza de manera adecuada, al tener el sistema discretizado su implementación en código está dada como la diferencia del error en un intervalo de tiempo. El diagrama de bloques quedaría de la siguiente manera:

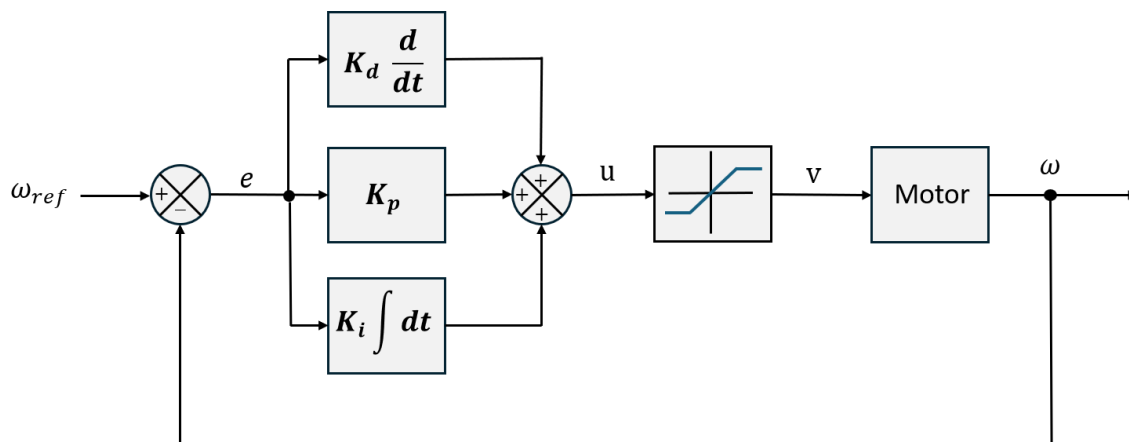


Figura 1.2 – Diagrama de bloques del controlador PI de velocidad de un motor DC.

El controlador PID procesa la señal de error e en tres ramas en paralelo: proporcional K_p , integral $K_i \int dt$ y derivativa $K_d \frac{d}{dt}$. La suma de estas tres acciones genera el esfuerzo de control u , el cual ingresa al bloque de saturación antes de alimentar al motor.

Un problema que enfrenté al trabajar con velocidad fue la calidad de la medición. Mi método calculaba la velocidad como la diferencia de pulsos entre dos lecturas consecutivas dividida entre el período de muestreo de 20 milisegundos. Dado que el *encoder* tiene una resolución de 2024 pulsos por revolución, la mínima variación detectable en un intervalo de 20 ms es un solo pulso, lo que equivale a saltos de 50 pulsos por segundo en la medición de velocidad. Es decir, la velocidad medida no variaba de forma continua, sino en escalones discretos de 50 pps. Esto introducía ruido

y dificultaba la estabilidad, especialmente cuando se requerían *setpoints* intermedios. Para resolverlo, implementé un filtro exponencial móvil o EMA, por sus siglas en inglés. Es un filtro discreto, pasa bajo que da más peso a los datos recientes al descontar los datos antiguos de forma exponencial. Este filtro suaviza la señal a costa de un pequeño retraso y es rápido, eficiente en memoria y fácil de implementar [8]. La lógica es sencilla: la nueva velocidad filtrada se calcula como un ponderado entre la medición actual y la velocidad filtrada anterior.

$$y_i = \alpha x_i + (1 - \alpha)y_{(i-1)}$$

Donde: y_i representa la salida filtrada en la muestra actual. x_i corresponde a la medición del sensor sin filtrar. y_{i-1} indica la estimación previa almacenada en memoria. α especifica la constante de ponderación del filtro. i hace referencia al paso o instante de muestreo. De esta forma, el retraso introducido es mínimo pero la señal de velocidad se vuelve mucho más suave, eliminando los escalones bruscos y permitiendo alcanzar *setpoints* intermedios que antes eran imposibles de estabilizar.

Generador de trayectorias punto a punto con perfil trapezoidal de velocidad y perfil parabólico - lineal - parabólico de posición.

Una vez que el lazo cerrado de velocidad funcionaba de manera estable con su controlador PID más filtro EMA, el siguiente paso fue generar consignas de velocidad que evolucionaran de forma suave en el tiempo. En [9] se explica que un movimiento punto a punto es aquel que acelera una carga, desde un punto de parada, hasta una velocidad constante y luego desacelera de tal forma que la velocidad y aceleración sean cero al alcanzar la distancia programada. Para ello, diseñé un generador de trayectorias basado en un perfil trapezoidal de velocidad. Este perfil está parametrizado por tres valores: la distancia total a recorrer X_{total} , la velocidad crucero deseada V_{max} y la aceleración máxima permitida a_{max} . A partir de estos parámetros, el generador calcula las ecuaciones horarias de la velocidad en cada fase del movimiento.

Formulación matemática del perfil trapezoidal de velocidad.

Primero es necesario determinar si la distancia es suficiente para alcanzar la velocidad crucero. La distancia mínima que requiere el motor para acelerar desde cero hasta V_{max} y luego desacelerar de vuelta a cero está dada por [10]:

$$X_{min} = \frac{V_{max}^2}{a_{max}}$$

Si $X_{total} > X_{min}$, el perfil es trapezoidal completo con tres fases: aceleración constante, velocidad de cruce y desaceleración constantes. Si $X_{total} \leq X_{min}$, entonces el perfil es triangular: el motor acelera hasta una velocidad pico menor que V_{max} y desacelera inmediatamente sin fase de cruce.

Caso trapezoidal:

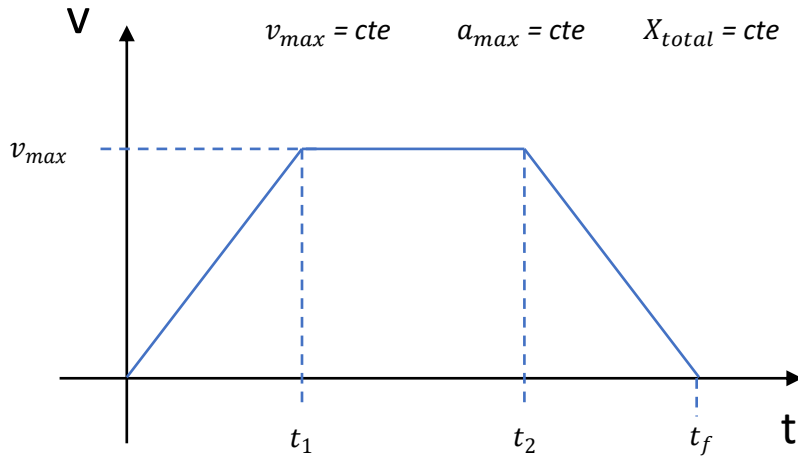


Figura 2 –Perfil trapezoidal de velocidad. Fases de aceleración, velocidad constante y desaceleración, con parámetros V_{max} , a_{max} , X_{total} y tiempos t_1 , t_2 , t_f .

El tiempo de transición entre la aceleración y velocidad máxima es:

$$t_1 = \frac{V_{max}}{a_{max}}$$

El tiempo en que se termina la etapa de velocidad máxima y comienza a desacelerar:

$$t_2 = \frac{X_{total}}{V_{max}}$$

Y el tiempo final del movimiento es:

$$t_f = t_1 + t_2$$

Las leyes de velocidad para cada etapa, en base a una deducción analítica propia construida con MRUA estándar, son:

$$V(t) = \begin{cases} a_{\max} t & 0 \leq t < t_1 \\ V_{\max} & t_1 \leq t < t_2 \\ V_{\max} - a_{\max}(t - t_2) & t_2 \leq t < t_f \\ 0 & t \geq t_f \end{cases}$$

O bien como sugiere Chen. [10] de manera elegante:

$$V(t) = \begin{cases} a_{\max} t & 0 \leq t < t_1 \\ a_{\max} t_1 & t_1 \leq t \leq t_f \\ a_{\max}(t_f - t) & t_2 \leq t < t_f \end{cases}$$

Caso triangular:

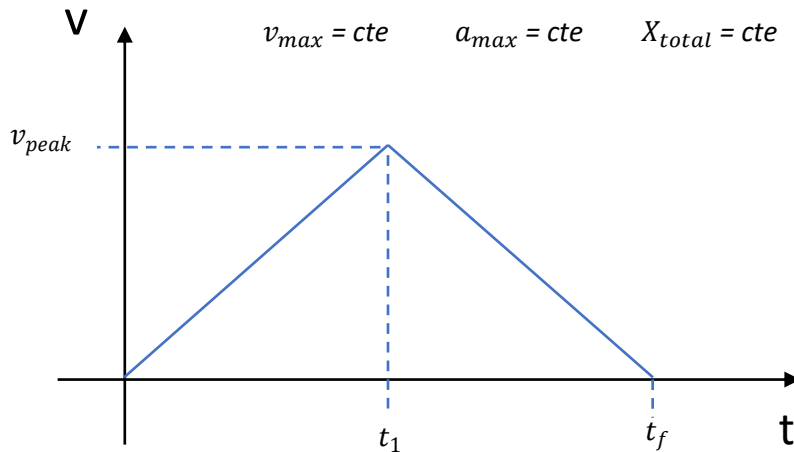


Figura 2.1 – Perfil triangular de velocidad. Fases de aceleración y desaceleración, sin etapa de velocidad constante, con parámetros V_{peak} , $a_{\max}$, X_{total} y tiempos t_1 y t_f .

En este caso la velocidad máxima alcanzable, llamada V_{peak} , es menor que $V_{\max}$ y se calcula como:

$$V_{\text{peak}} = \sqrt{a_{\max} X_{\text{total}}}$$

El tiempo de transición entre la aceleración y la desaceleración es:

$$t_1 = \frac{V_{\text{peak}}}{a_{\text{max}}}$$

El tiempo final es:

$$t_f = 2t_1$$

Y la velocidad en función del tiempo queda:

$$V(t) = \begin{cases} a_{\text{max}} t & 0 \leq t < t_1 \\ V_{\text{peak}} - a_{\text{max}}(t - t_1) & t_1 \leq t \leq t_f \\ 0 & t \geq t_f \end{cases}$$

Siguiendo con la formulación de Chen [10]:

$$V(t) = \begin{cases} a_{\text{max}} t & 0 \leq t < t_1 \\ a_{\text{max}}(t_f - t) & t_1 \leq t \leq t_f \end{cases}$$

Con estas ecuaciones, el generador puede producir en cada instante la consigna de velocidad que el motor debe seguir.

Para la generación del perfil trapezoidal en el código, separé los cálculos pesados de la ejecución en tiempo real. En primer lugar, definí una función que se ejecuta una sola vez al iniciar el programa o cuando se solicita un nuevo desplazamiento. Esta función recibe como parámetros la distancia total a recorrer, la velocidad crucero deseada y la aceleración máxima permitida. Dentro de ella, calculo la distancia mínima necesaria para alcanzar la velocidad crucero como el cuadrado de la velocidad dividido entre la aceleración. Comparo ese valor con el valor absoluto de la distancia solicitada. Si la distancia es mayor o igual a la mínima, determino que el perfil es trapezoidal completo. En ese caso, calculo el tiempo de aceleración como la velocidad crucero dividida entre

la aceleración, el tiempo de velocidad constante como la distancia absoluta dividida entre la velocidad crucero, y el tiempo total como la suma de ambos. Si la distancia es

menor que la mínima, el perfil es triangular: calculo la velocidad pico alcanzable como la raíz cuadrada del producto de la distancia absoluta por la aceleración, luego el tiempo de aceleración como esa velocidad pico dividida entre la aceleración, y el tiempo total como el doble de ese tiempo. Todos estos tiempos se almacenan para ser usados después, junto con una bandera que indica si el perfil es triangular o trapezoidal.

La generación del *setpoint* dinámico ocurre dentro de una rutina de interrupción configurada para ejecutarse cada 10 milisegundos. Dentro de esta rutina, incremento un contador y calculo el tiempo transcurrido multiplicando el número de interrupciones transcurridas por el período de muestreo. Luego reviso la bandera del tipo de perfil. Si es trapezoidal, comparo el tiempo transcurrido con los tiempos de aceleración, de velocidad constante y total. Según el tramo en que se encuentre, calculo el *setpoint* instantáneo como aceleración por tiempo durante la rampa de subida, como velocidad crucero constante durante la fase central, o como velocidad crucero decreciente durante la rampa de bajada. Si el tiempo supera el total, el *setpoint* se fuerza a cero. Si el perfil es triangular, uso los tiempos de aceleración y total junto con la velocidad pico para calcular el *setpoint* de forma similar, pero sin fase de velocidad constante. Este *setpoint* dinámico se guarda en una variable global compartida, que la interrupción del lazo de velocidad, ejecutándose cada 10 milisegundos, lee como su consigna de referencia. De esta manera, el generador de trayectorias y el controlador de velocidad trabajan de forma independiente y sincronizada

Formulación matemática del perfil asociado de posición con desplazamiento lineal

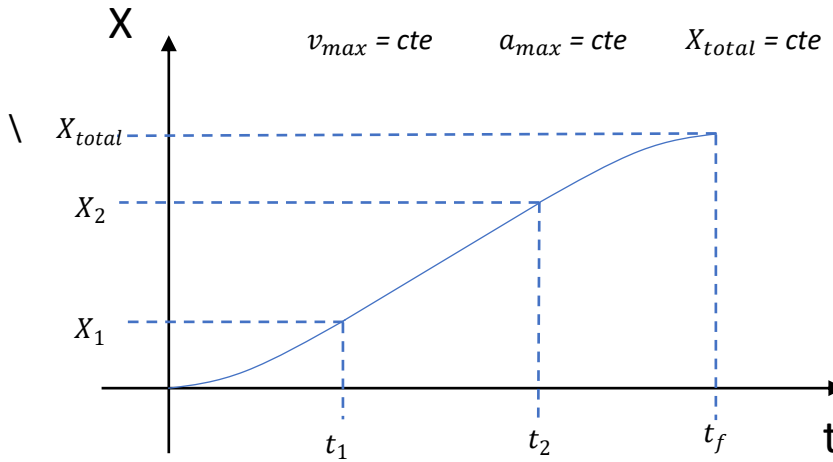


Figura 2.2 – Perfil parabólico - lineal -parabólico de posición. Con parámetros V_{max} , a_{max} , X_{total} y tiempos t_1 , t_2 , t_f .

Donde las distancias recorridas son:

$$x_1 = \frac{V_{max}^2}{2a_{max}} \quad x_2 = x_{total} - x_1$$

Siguiendo con los mismos tiempos de transición, al integrar las ecuaciones de la velocidad se deduce:

$$X(t) = \begin{cases} \frac{1}{2} a_{max} t^2 & 0 \leq t < t_1 \\ x_1 + V_{max} (t - t_1) & t_1 \leq t < t_2 \\ x_2 + V_{max} (t - t_2) - \frac{1}{2} a_{max} (t - t_2)^2 & t_2 \leq t < t_f \\ x_{total} & t \geq t_f \end{cases}$$

Chen [10] muestra una forma más elegante y optima computacionalmente:

$$X(t) = \begin{cases} \frac{1}{2} a_{\max} t^2 & 0 \leq t < t_1 \\ \frac{1}{2} a_{\max} t_1 (2t - t_1) & t_1 \leq t < t_2 \\ x_f - \frac{1}{2} a_{\max} (t_f - t)^2 & t_2 \leq t < t_f \end{cases}$$

Perfil parabólico de posición

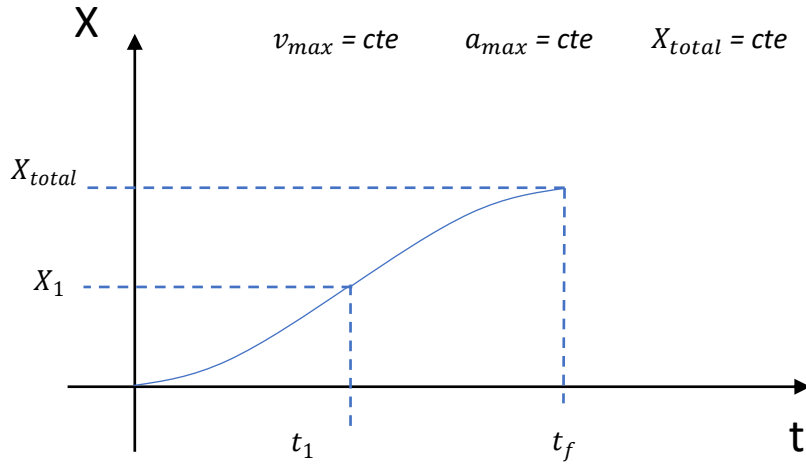


Figura 2.3 – Perfil parabólico de posición. Con parámetros $V_{\max}$, $a_{\max}$, X_{total} y tiempos t_1 , t_2 , t_f .

La función de la posición en base al tiempo al no alcanzar la velocidad cruce:

$$X(t) = \begin{cases} \frac{1}{2} a_{\max} t^2 & 0 \leq t < t_1 \\ x_1 + V_{\text{peak}} (t - t_1) - \frac{1}{2} a_{\max} (t - t_1)^2 & t_1 \leq t \leq t_f \\ x_{\text{total}} & t \geq t_f \end{cases}$$

La implementación de las ecuaciones de posición no significó problema en el código, al ya contar con la lógica que define cuando existe una fase con velocidad cruce y tener

el sistema discretizado solo fue cuestión de introducir las fórmulas junto a las de la velocidad

Controlador de doble retroalimentación para seguimiento de posición

Para lograr un control preciso de la posición del motor, se implementó una estrategia de **doble retroalimentación** que aprovecha simultáneamente la información de la posición y la velocidad del eje. Este enfoque permite corregir no solo el error de posición, sino también la rapidez con la que ese error se reduce, mejorando significativamente la respuesta transitoria y el amortiguamiento del sistema.

La arquitectura del controlador se basa en dos lazos anidados:

- **Lazo externo de posición:** Compara la posición deseada, proporcionada por el generador de trayectoria, con la posición real medida por el *encoder*. El error de posición se define como:

$$e_p(t) = p_{\text{ref}}(t) - p_{\text{real}}(t)$$

- **Lazo interno de velocidad:** Compara la velocidad de referencia, también proporcionada por el generador de trayectoria, con la velocidad real del motor, la cual se obtiene a partir de la derivada de la posición del *encoder* y se filtra mediante un *EMA (Exponential Moving Average)* para reducir el ruido. El error de velocidad es:

$$e_v(t) = v_{\text{ref}}(t) - v_{\text{real}}(t)$$

La señal de control final que se aplica al motor, a través del *PWM*, es la suma ponderada de ambos errores, cada uno multiplicado por su respectiva ganancia:

$$u(t) = K_p \cdot e_p(t) + K_d \cdot e_v(t)$$

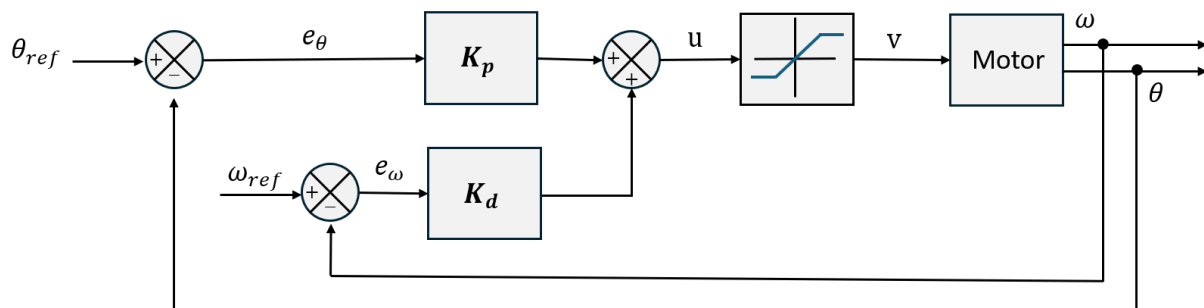


Figura 3 – Diagrama de bloques del controlador de posición con doble retroalimentación.

En la figura 3 se ve gráficamente como el sistema cuenta con dos puntos de suma correspondientes a la posición y velocidad de referencia provenientes del generador de trayectoria punto a punto los cuales se comparan con la velocidad y posición real del motor.

Sincronización de motores mediante factor de escala.

Cada motor i dispone de sus propios parámetros nominales:

- Velocidad máxima: $V_{max,i}$
- Aceleración máxima: $a_{max,i}$
- Distancia total para recorrer: d_i

Biagiotti [11] explica que el objetivo es que ambos motores completen su movimiento en el mismo tiempo total t_f , respetando las capacidades dinámicas de cada actuador. La solución adoptada consiste en identificar el motor con la mayor distancia y utilizarlo como referencia, ya que requiere el mayor tiempo si se mantienen sus parámetros originales. El otro motor, al tener que recorrer una distancia menor, debe reducir sus velocidades y aceleraciones de forma proporcional para que su tiempo total se iguale al del motor de referencia.

Factor de escala y ajuste de parámetros.

Se define el factor de escala k como la relación entre la distancia menor y la mayor:

$$k = \frac{\min(d_1, d_2)}{\max(d_1, d_2)} \quad (0 < k \leq 1)$$

El motor que debe recorrer la distancia mayor conserva sus valores originales de V_{max} y a_{max} . El motor con la distancia menor escala sus parámetros de la siguiente manera:

$$\begin{aligned} V'_{max} &= k \cdot V_{max} \\ a'_{max} &= k \cdot a_{max} \end{aligned}$$

donde V'_{max} y a'_{max} son los valores efectivos que utilizará el motor de menor distancia. Esta escala lineal garantiza que la duración total del movimiento se ajuste a la del motor de referencia.

Tercera Etapa

En esta etapa se trabajó en el modelo cinemático de un robot diferencial con la finalidad de poder moverlo a una coordenada deseada en un espacio libre de colisiones.

Modelo cinemático de un robot diferencial.

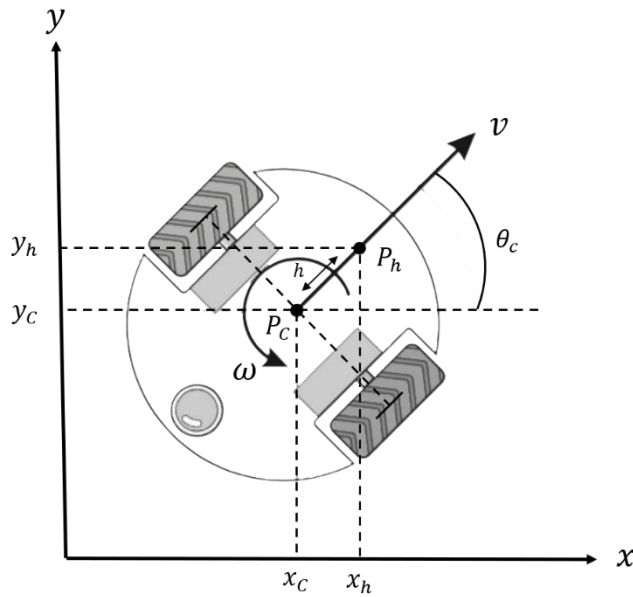


Figura 4. Esquema geométrico del robot móvil diferencial en el marco de referencia global.

El modelo cinemático del punto central

$$\begin{aligned}\dot{x}_c &= v \cos \theta \\ \dot{y}_c &= v \sin \theta \\ \dot{\theta} &= w\end{aligned}$$

En forma matricial separando el vector de estado global del vector de velocidades locales del chasis:

$$\begin{bmatrix} \dot{x}_c \\ \dot{y}_c \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ w \end{bmatrix}$$

Al tratarse de una matriz no cuadrada, no existe una matriz inversa, lo cual refleja matemáticamente la restricción no holonómica del robot diferencial. Debido a esta limitación algebraico-geométrica, esta formulación no se utiliza para el control

cinemático inverso de trayectoria de forma directa. Sin embargo, este modelo es fundamental para la odometría del robot.

Modelo cinemático del punto h

$$\dot{x}_h = V \cos \theta - h\omega \sin \theta$$

$$\dot{y}_h = V \sin \theta + h\omega \cos \theta$$

$$\dot{\theta} = \omega$$

En forma matricial

$$\begin{bmatrix} \dot{x}_h \\ \dot{y}_h \end{bmatrix} = \underbrace{\begin{bmatrix} \cos \theta & -h \sin \theta \\ \sin \theta & h \cos \theta \end{bmatrix}}_{J_h} \begin{bmatrix} V \\ \omega \end{bmatrix}$$

Para despejar las velocidades de control del chasis $[V, \omega]^T$, debemos invertir la matriz jacobiana.

$$\begin{bmatrix} V \\ \omega \end{bmatrix} = J_h^{-1} \begin{bmatrix} \dot{x}_h \\ \dot{y}_h \end{bmatrix}$$

$$J_h^{-1} = \frac{1}{h} \begin{bmatrix} h \cos \theta & h \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}$$

Modelo Cinemático Inverso Resultante.

$$\begin{bmatrix} V \\ \omega \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\frac{\sin \theta}{h} & \frac{\cos \theta}{h} \end{bmatrix} \begin{bmatrix} \dot{x}_h \\ \dot{y}_h \end{bmatrix}$$

Control cinemático.

Considerando que se busca regular la posición del robot para alcanzar un punto de referencia deseado (x_{hd}, y_{hd}) , se propone sustituir las velocidades $(\dot{x}_h, \dot{y}_h)$ por las señales de control (u_x, u_y) :

$$\begin{bmatrix} V \\ \omega \end{bmatrix} = J^{-1} \begin{bmatrix} u_x \\ u_y \end{bmatrix}$$

Donde las acciones de control u_x y u_y corresponden a la regulación del error de posición con una ganancia $k > 0$:

$$u_x = k \cdot e_x = k(x_{hd} - x_h)$$

$$u_y = k \cdot e_y = k(y_{hd} - y_h)$$

Para lograr que el robot siga una trayectoria variante en el tiempo $(x_{hd}(t), y_{hd}(t))$, se incorporan las velocidades deseadas de la referencia $(\dot{x}_{hd}, \dot{y}_{hd})$ a las señales de control u_x y u_y :

$$u_x = \dot{x}_{hd} + k(x_{hd} - x_h)$$

$$u_y = \dot{y}_{hd} + k(y_{hd} - y_h)$$

Finalmente obtenemos:

$$V = u_x \cos \theta + u_y \sin \theta$$

$$\omega = -\left(\frac{u_x}{h}\right) \sin \theta + \left(\frac{u_y}{h}\right) \cos \theta$$

Este modelo proporciona las velocidades lineales y angulares necesarias para cumplir con la consigna de navegación. Sin embargo, para su ejecución real, requerimos el modelo cinemático inverso que permita mapear estas velocidades (v, ω) a las velocidades angulares de cada motor (ω_m) , considerando la geometría del robot y las restricciones mecánicas (como el radio de las ruedas y la distancia entre ellas).

Modelo cinemático directo con marco local

Este modelo establece la relación entre la velocidad lineal (V) y angular (ω) del chasis del robot en su marco de referencia local y las velocidades angulares de las ruedas derecha (ω_R) e izquierda (ω_L).

Sabiendo que las velocidades tangenciales de las ruedas dependen del radio r :

$$V_R = r \cdot \omega_R$$

$$V_L = r \cdot \omega_L$$

Las velocidades del chasis se expresan como:

$$V = \frac{V_R + V_L}{2} = \frac{r(\omega_R + \omega_L)}{2}$$

$$\omega = \frac{V_R - V_L}{L} = \frac{r(\omega_R - \omega_L)}{L}$$

Representación matricial separando las velocidades angulares de los motores en forma de vector:

$$\begin{bmatrix} V \\ \omega \end{bmatrix} = \underbrace{\begin{bmatrix} \frac{r}{2} & \frac{r}{2} \\ r & -r \\ \frac{L}{2} & -\frac{L}{2} \end{bmatrix}}_A \begin{bmatrix} \omega_R \\ \omega_L \end{bmatrix}$$

Para despejar las velocidades angulares de las ruedas $[\omega_R, \omega_L]^T$, debemos invertir la matriz A .

$$\begin{bmatrix} \omega_R \\ \omega_L \end{bmatrix} = A^{-1} \begin{bmatrix} V \\ \omega \end{bmatrix}$$

$$A^{-1} = -\frac{L}{r^2} \begin{bmatrix} -\frac{r}{L} & -\frac{r}{2} \\ \frac{r}{L} & \frac{r}{2} \end{bmatrix}$$

Modelo cinemático inverso resultante

$$\begin{bmatrix} \omega_R \\ \omega_L \end{bmatrix} = \begin{bmatrix} \frac{1}{r} & \frac{L}{2r} \\ \frac{1}{r} & -\frac{L}{2r} \end{bmatrix} \begin{bmatrix} V \\ \omega \end{bmatrix}$$

Ecuaciones escalares

$$\omega_R = \frac{2V + L\omega}{2r}$$

$$\omega_L = \frac{2V - L\omega}{2r}$$

Robot diferencial con perfil trapezoidal de velocidad y telemetría.

Antes de implementar la cinemática del robot implementé los generadores de trayectorias punto a punto que estuve trabajando a lo largo de la estadía. Si ambos motores siguen el mismo perfil el robot avanzara en línea recta hasta alcanzar una determinada distancia presentando etapas de aceleración y desaceleración, cabe mencionar que esta aplicación no tomaría en cuenta su orientación por lo que para trayectoria rectas el robot podría presentar desvíos en distancias considerables. Este paso permitió ajustar y resolver distintos problemas de electrónica y de telemetría antes de poder fusionar los perfiles con los modelos cinemáticos.

La implementación de mis códigos de sincronización de perfiles trapezoidales para el robot no represento gran reto ni modificaciones significativas, pero me vi en la necesidad de poder comunicarme con el microcontrolador de manera remota. Hay distintas formas de realizar telemetría con la ESP32 dependiendo de la vía y el protocolo. Como vía de comunicación use Wifi y utilice un protocolo llamado *WebSockets* que establece un canal bidireccional y en tiempo real entre un cliente web y un servidor. A diferencia de la arquitectura HTTP convencional, la cual requiere refrescar la página o realizar peticiones periódicas para actualizar la información, *WebSocket* mantiene una conexión persistente. Esto permite un intercambio instantáneo de datos sin necesidad de recargar la interfaz, siendo fundamental en aplicaciones de alta reactividad como mensajería instantánea, videojuegos en línea y monitoreo de variables en tiempo real [12].

Para la implementación de la telemetría se emplearon tres librerías en cadena de dependencia:

WiFi.h → AsyncTCP → ESPAsyncWebServer

- **WiFi.h:** Gestión de la conexión del ESP32 a la red local en modo Estación (STA).
- **AsyncTCP:** Capa de soporte para el manejo de sockets TCP de forma asíncrona y no bloqueante.
- **ESPAsyncWebServer:** Construcción del servidor HTTP (puerto 80) y del punto de enlace (*endpoint*) *WebSocket* en la ruta */ws*.

Flujo de Operación

1. El ESP32 se conecta a la red Wi-Fi y habilita el servidor web con el socket en /ws.
2. El cliente abre la página web y establece un canal bidireccional mediante eventos (conexión, desconexión y recepción).
3. La telemetría opera bajo un esquema *push* (difusión/broadcast) en tiempo real, donde el ESP32 transmite datos periódicamente sin necesidad de recibir solicitudes HTTP constantes (*polling*).

Nota aclaratoria: El diseño de la interfaz gráfica en HTML, la selección del conjunto de librerías y la estructura de implementación del código fueron generados con la asistencia del modelo de inteligencia artificial Claude.

Cuarta Etapa.

Esta etapa abarca la materialización física del robot móvil diferencial a través de herramientas de diseño asistido por computadora como SolidWorks, manufactura aditiva por una impresora 3D modelo Ender V3 SE. El objetivo principal es contar con una plataforma mecánica y electrónicamente robusta para llevar a cabo la fase de instrumentación y validación del control cinemático.

Concepción Mecánica y Diseño en CAD

El diseño del robot está inspirado en las plataformas móviles de grado industrial orientadas a la logística e intralogística, donde la característica primordial es ofrecer una superficie totalmente plana en su cubierta superior para el transporte o montaje de cargas de trabajo y módulos adicionales.



Figura 5. Robot Freight



Figura 5.1. Robot Freight

Principios de Diseño y Arquitectura:

1. **Geometría Circular:** Se seleccionó una geometría cilíndrica/disco para el chasis. Esta configuración favorece la cinemática diferencial, permitiendo giros sobre su propio eje sin colisionar esquinas y distribuyendo el centro de masa de manera homogénea.
2. **Operatividad Funcional Simplificada.** La placa base principal contiene la totalidad de los soportes y anclajes mecánicos para albergar la electrónica, los actuadores y la batería. Por ello, el robot es 100% operacional en su estado básico (solo con la placa base inferior), sin depender de la presencia de paredes o tapa superior para ejecutar pruebas de movimiento o control.
3. **Mecanismo de Acople de Paredes y Tapa:** La base cuenta con una serie de pestañas y ranuras de ensamble dispuestas en su periferia. Estas permiten adaptar paredes laterales de protección y, finalmente, colocar la cubierta plana superior cuando se requiera el cerramiento completo del prototipo.

Diseño CAD

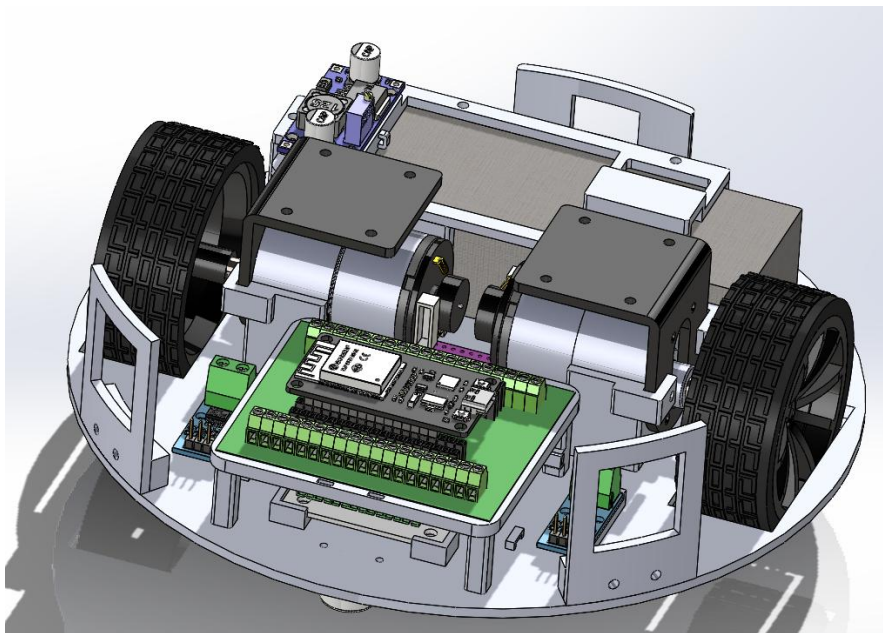


Figura 6. Diseño CAD en SolidWorks del robot diferencial

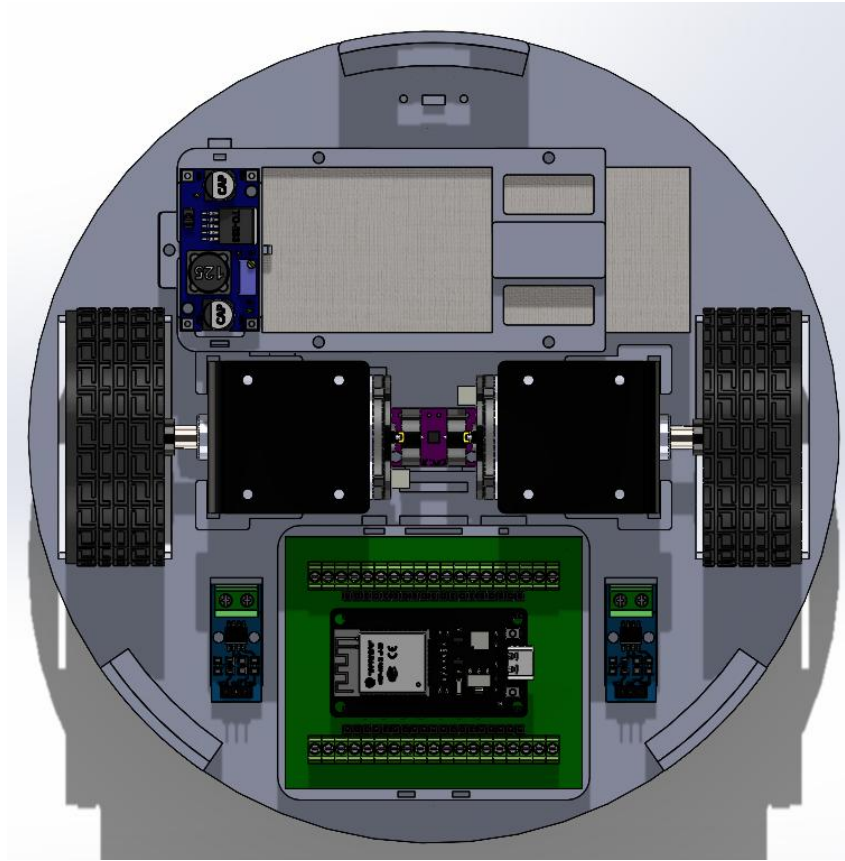


Figura 6.1 Diseño CAD en SolidWorks del robot diferencial vista superior

El chasis del robot presenta un diámetro de 21.5 cm y una altura libre al suelo de aproximadamente 1.4 cm. La altura mínima disponible para la ubicación de la tapa superior, medida desde la base del chasis, es de 6 cm. La placa base incorpora canales especialmente diseñados para el tendido y organización del cableado, así como las perforaciones y alojamientos necesarios para el montaje de cada componente en su posición correspondiente. Adicionalmente, la unidad de medición inercial se encuentra dispuesta en el centro geométrico del chasis, asegurando una referencia de medición óptima para los algoritmos de control y navegación. Se aprovecharon los *brackets* de los motores para dar soporte cuando se requiera montar su tapa superior.

Tabla 3. Listado del material empleado

<i>Nombre</i>	<i>Nombre Técnico / Modelo</i>	<i>Descripción</i>	<i>Cantidad</i>	<i>Precio unitario</i>
<i>Microcontrolador</i>	ESP32-WROOM-32D con placa de terminales	Unidad central de procesamiento.	1	\$150
<i>Motores DC</i>	Kit JGB37-520 (12V, 200 RPM con Encoder, ruedas y brackets	Actuadores principales de tracción diferencial.	2	\$250
<i>Driver de Motores</i>	TB6612FNG	Controlador dual en puente H para gestión de PWM y sentido de giro.	1	\$40
<i>Sensores de Corriente</i>	ACS712 Módulo de lectura hasta 5A	Medición de consumo eléctrico por efecto Hall en cada motor.	2	\$20
<i>Sensor IMU</i>	GY-BNO080 / BNO085 (9DOF)	Unidad de medición inercial para sensado de orientación y rotación.	1	\$210
<i>Rueda de Apoyo</i>	Rueda esférica Mini 3PI (Transferencia de bolas N20)	Punto de apoyo omnidireccional para equilibrio del chasis.	2	\$30
<i>Regulador de Voltaje</i>	Módulo DC-DC Buck LM2596	Convertidor reductor de voltaje para alimentar los módulos.	1	\$45
<i>Batería</i>	LiPo 3S 11.1V 5200 mAh (Conector XT60)	Fuente principal de alimentación de potencia y control.	1	\$700
<i>Filamento</i>	Filamento PETG Creality 1kg	Material para la impresión de la base circular del robot.	1	\$300

En la tabla 3 se aprecian los materiales requeridos para el armado del robot los cuales fueron pedidos por AliExpress, la entrega de los productos tardó entre 1 a 2 semanas. Cabe mencionar que ya contaba con algunos materiales como la LiPo o la ESP32 con su placa con terminales t-block sin embargo se hizo su respectiva cotización dando así un

total de 2000 pesos aproximadamente. Se emplearon herramientas como pinzas de corte, estación de soldadura, pinzas de precisión, destornilladores, herramienta rotativa Dremel, además del uso de tornillería milimétrica y *termofits* con los que contaba en mi unida

Resultados

En este apartado se presentan los resultados obtenidos durante el desarrollo del proyecto, organizados de acuerdo con las etapas de la metodología. Cada resultado demuestra el cumplimiento de los objetivos operativos correspondientes y se apoya en fragmentos de código, gráficas, imágenes y descripciones técnicas.

Etapas 1:

Acondicionamiento del hardware y software base

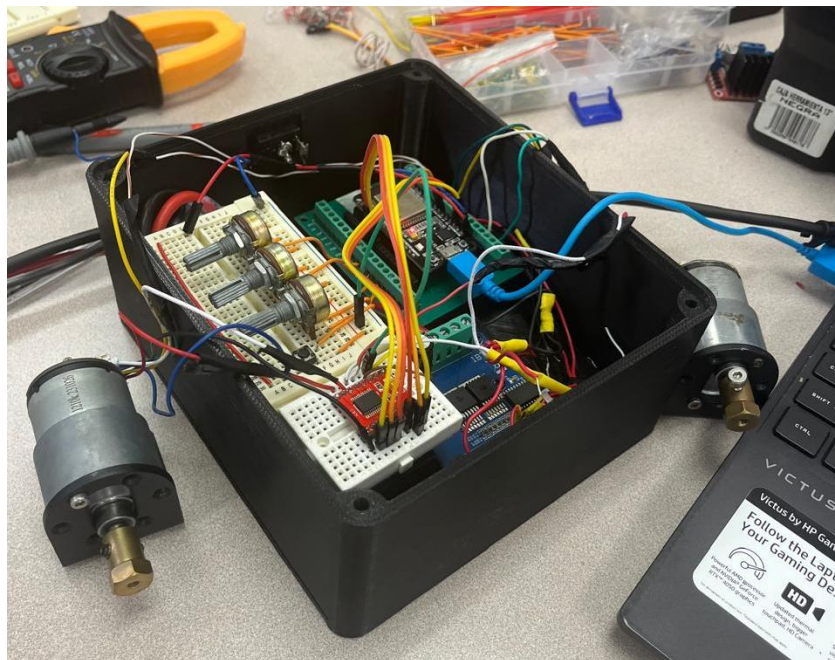


Figura 7 – Banco de pruebas para los motores DC

En la figura 7 se muestra el banco en el cual realice mis primeras prácticas y pruebas con la finalidad de dominar el hardware y software a utilizar. El banco contaba con los *drivers*

para motores TB6612, IBT-2 y el L298N, una placa para la ESP32, potenciómetros, botones, LiPo de 12v y un regulador DC-DC.

La primera etapa concluyó exitosamente con la instrumentación completa del banco de pruebas. Se logró:

- Configurar el módulo PWM del ESP32 a 20 kHz con resolución de 11 bits, validando que el divisor de reloj resultante (1.95) se encuentra dentro del rango permitido por el fabricante.
- Caracterizar el umbral mínimo de PWM para vencer la fricción estática del motor reductor, obteniendo un valor de 370 sobre 2047 (aproximadamente 18% del ciclo de trabajo).
- Implementar la lectura del *encoder* incremental en cuadratura completa utilizando el periférico *PCNT* a través de la librería ESP32Encoder, alcanzando una resolución efectiva de 2024 pulsos por revolución.
- Desarrollar las funciones de control básico tales como *PWM*, *ADC*, interrupciones externas e internas y verificar la comunicación con el Serial Plotter para visualización en tiempo real.

Estos logros dejaron el sistema listo para implementar estrategias de control en lazo cerrado.

Etapas 2:

Control de posición con controlador proporcional

Como primer paso del control en lazo cerrado, se implementó un controlador puramente proporcional (P) para regular la posición angular del motor. El código se

ejecuta dentro de una interrupción por temporizador configurada cada 20 milisegundos, lo que garantiza un período de muestreo constante. A continuación, se muestra el fragmento principal de la rutina de control.

Código 1 – Rutina de interrupción para control P de posición.

```
“void IRAM_ATTR P_control() {  
  
    portENTER_CRITICAL_ISR(&mux);  
    int64_t c = encoder.getCount();  
    portEXIT_CRITICAL_ISR(&mux);  
  
    float position = (float)c;  
    float error    = SETPOINT_PULSOS -  
    position;  
  
    // P  
    float output = KP * error;  
  
    // Saturación PWM  
    int abs_pwm = (int32_t)fabsf(output);  
    if (abs_pwm < PWM_MIN) abs_pwm = PWM_MIN;  
    if (abs_pwm > PWM_MAX) abs_pwm = PWM_MAX;  
  
    //Determinar sentido  
    if (output > 0) {  
        ledcWrite(CH_RPWM, 0);  
        ledcWrite(CH_LPWM, abs_pwm);  
    }  
    else if (output < 0) {  
        ledcWrite(CH_LPWM, 0);  
        ledcWrite(CH_RPWM, abs_pwm);  
    }  
    else {  
        ledcWrite(CH_LPWM, 0);  
        ledcWrite(CH_RPWM, 0);  
    }  
  
    dbg_position = position;  
    dbg_error    = error;  
    dbg_output   = output;  
    dbg_pwm      = abs_pwm;  
}”
```

La consigna se fijó en 10000 pulsos, que corresponde aproximadamente a 4.94 revoluciones del eje de salida (considerando 2024 pulsos por vuelta). Las siguiente grafica muestra la evolución de la posición medida por el *encoder* frente al *setpoint*.

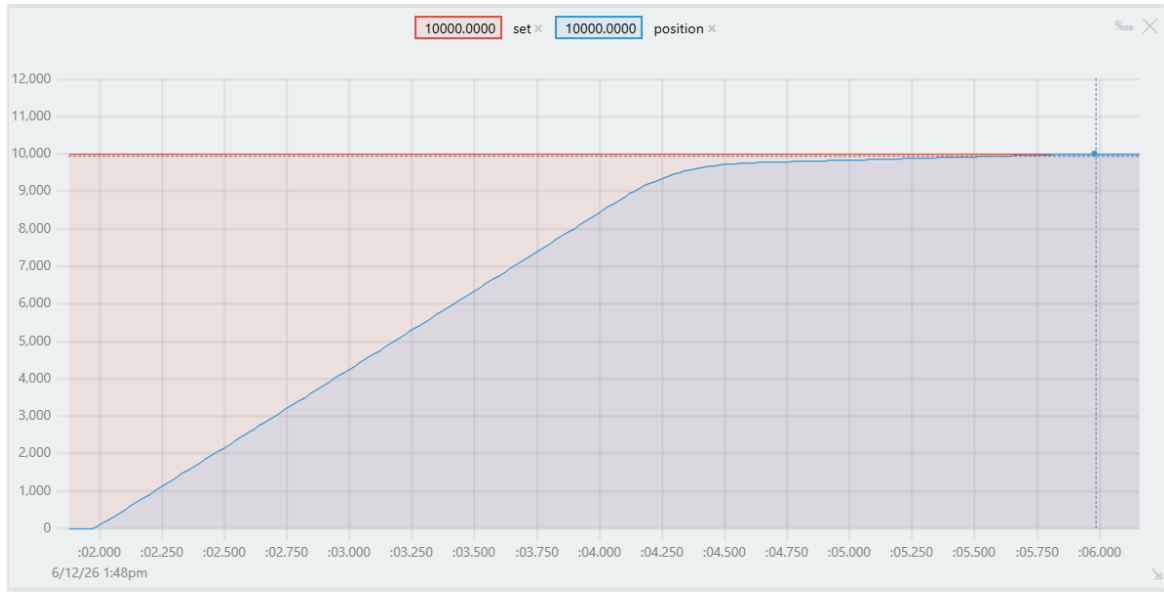


Figura 8 –Respuesta de posición ante consigna de 10000 pulsos con $K_p = 1.5$. la unidad de tiempo en el eje x son segundos.

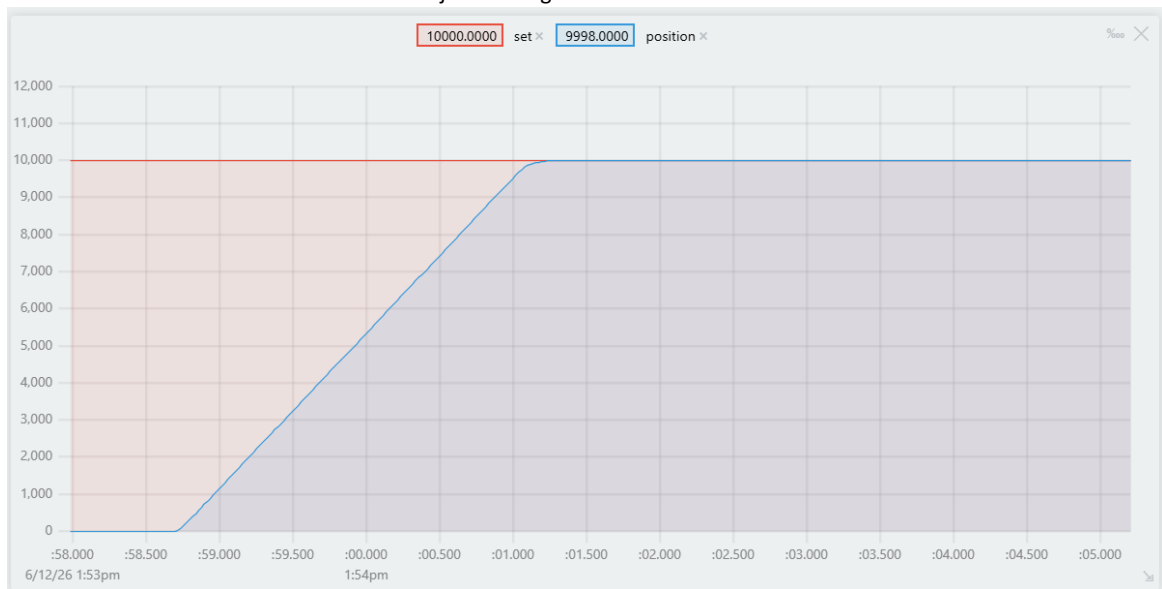


Figura 8.1 –Respuesta de posición ante consigna de 10000 pulsos con $K_p = 6.0$. La unidad de tiempo en el eje x son segundos.

En la figura 8.1 se observa la respuesta ante un escalón de 10000 pulsos con $K_p=6.0$. El sistema alcanza el valor de consigna por primera vez en aproximadamente 2.5 segundos, sin embargo, al carecer de zona muerta y debido a la alta ganancia proporcional, la posición presenta una oscilación mantenida de ± 4 pulsos, muy superior a la observada en la figura 3 en la cual tardo aproximadamente 6 segundos en llegar exactamente a la consigna.

Control de velocidad con controlador PID

El objetivo principal fue lograr que el motor mantuviera una velocidad constante (en pulsos por segundo) ante perturbaciones y cambios de carga, minimizando el error en estado estacionario mediante la acción integral.

Código 2 – Rutina de interrupción para control PID de velocidad.

```
“void IRAM_ATTR PID_control() {

    portENTER_CRITICAL_ISR(&mux);
    int64_t c = encoder.getCount();
    portEXIT_CRITICAL_ISR(&mux);

    //velocidad pulso/s
    float raw_velocity = (float)(c - last_count) / DT;
    last_count = c;

    //filtro EMA
    filtered_velocity = ALPHA * raw_velocity + (1-
    ALPHA) * filtered_velocity;

    //Error en pulsos/s
    float error = SETPOINT_PPS - filtered_velocity;

    //acumulacion de error (integral)
    integral_acc += error * DT;
    if (integral_acc > I_MAX) integral_acc = I_MAX;
    if (integral_acc < I_MIN) integral_acc = I_MIN;

    //limpia del acumulado en oversetpoint
    if (fabsf(SETPOINT_PPS) - fabsf(filtered_velocity)
    < -200) integral_acc = 0;

    //derivativo
    float derivative_term = (error - last_error)/DT;
    last_error = error;
    //float derivative_term = -(filtered_velocity -
    last_velocity)/DT;
    //last_velocity = filtered_velocity;
```

```
//PID
float output = (KP * error) + (KI * integral_acc)
+ (KD * derivative_term);

//saturacion pwm
int abs_pwm = (int32_t)fabsf(output);
if (abs_pwm < PWM_MIN) abs_pwm = PWM_MIN;
if (abs_pwm > PWM_MAX) abs_pwm = PWM_MAX;

if (output >= 0) {
    ledcWrite(CH_RPWM,0);
    ledcWrite(CH_LPWM,abs_pwm);
}
else {

    ledcWrite(CH_LPWM,0);
    ledcWrite(CH_RPWM, abs_pwm);
}
}”
```

El controlador se programó en una rutina de interrupción temporizada con período fijo $DT = 20$ ms. Para reducir el ruido inherente a la medición de velocidad derivada del *encoder*, se aplicó un filtro de media móvil exponencial (EMA) con coeficiente $\alpha = 0.3$. La señal de control resultante se saturó para limitar el ciclo de trabajo del PWM entre un mínimo (365) y un máximo (2047), asegurando así que el motor arranque con un pulso mínimo que supere el umbral de fricción estática.

Adicionalmente, se incorporaron mecanismos de anti-windup: la integral se satura entre ± 5000 y se reinicia a cero cuando la velocidad real supera el *setpoint* en más de 200 pulsos/s, evitando así sobreacumulación durante transitorios.

Filtración de velocidades

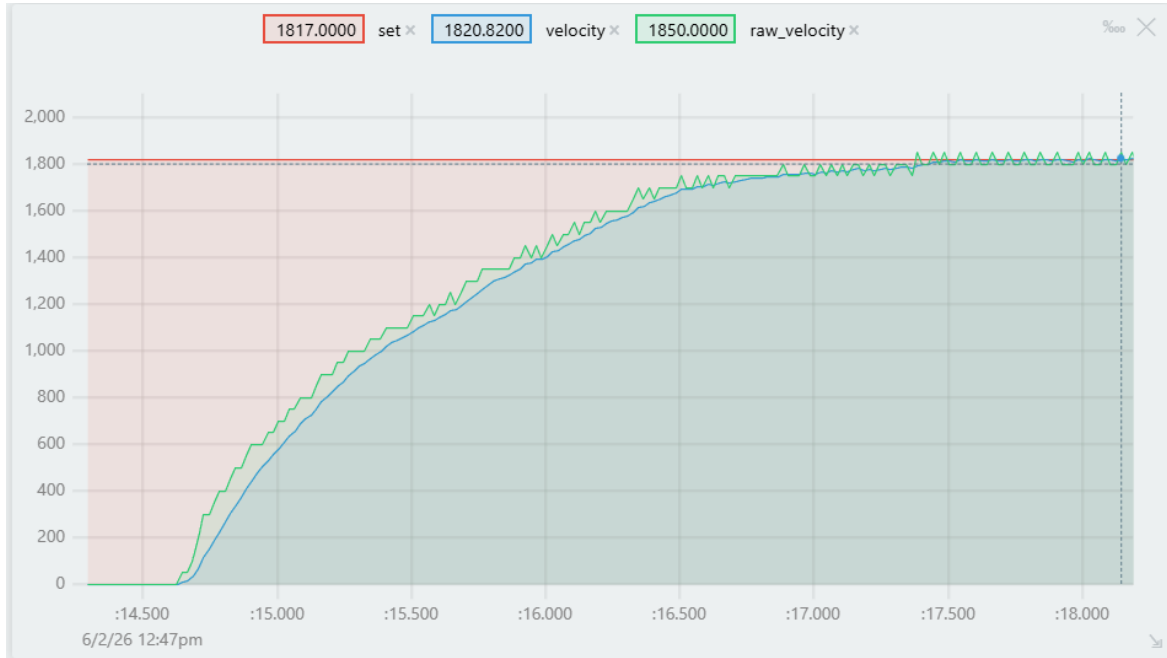


Figura 9 –Velocidad con filtro EMA con constante de suavidad de 0.2, la unidad de tiempo en el eje x son segundos.

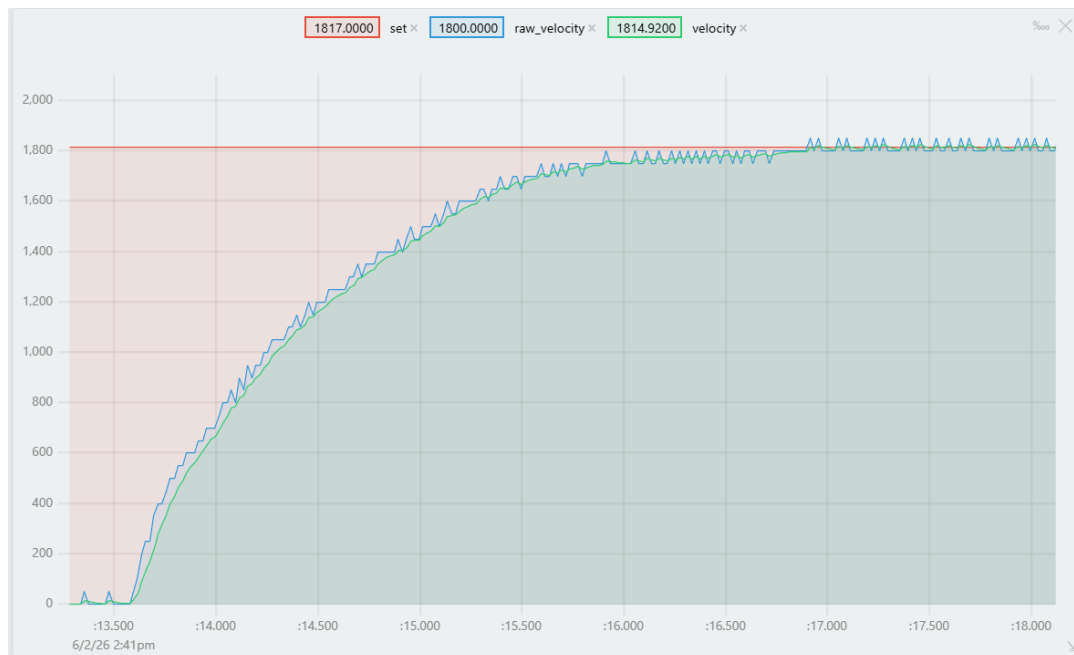


Figura 9.1 –Velocidad con filtro EMA con constante de suavidad de 0.3. La unidad de tiempo en el eje x son segundos.

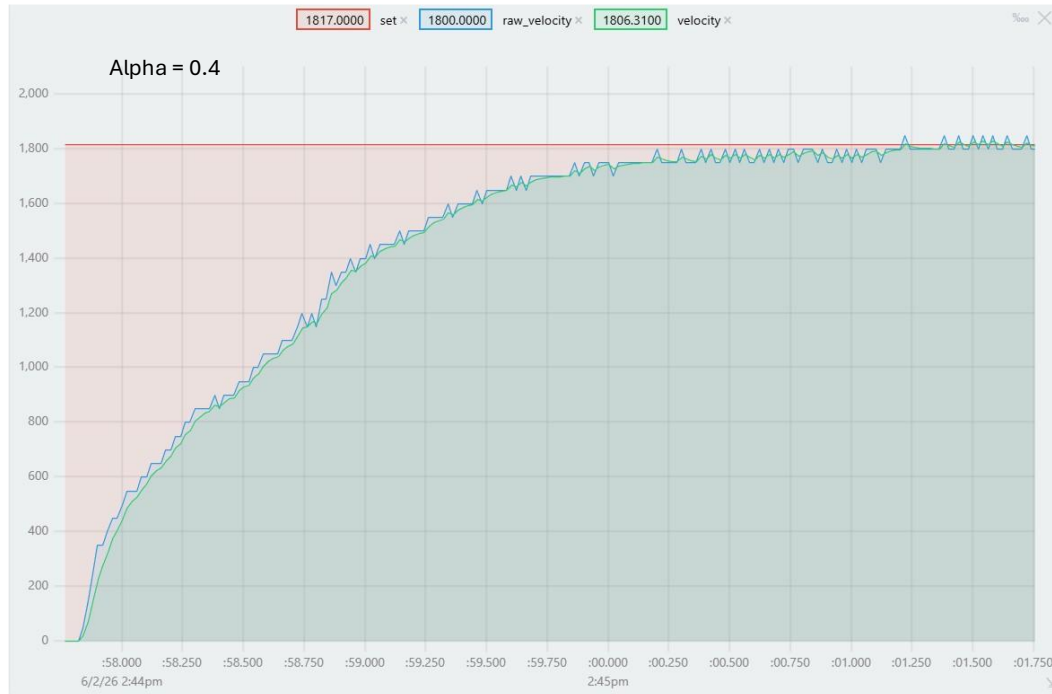


Figura 9.2 –Velocidad con filtro EMA con constante de suavidad de 0.4. La unidad de tiempo en el eje x son segundos.

La Figura 9.1 muestra la comparación entre la velocidad cruda y la velocidad filtrada mediante un filtro de media móvil exponencial (EMA) con coeficiente $\alpha = 0.3$, para un setpoint de 1800 pulsos/s. La rutina de control se ejecuta cada $DT = 20$ ms, lo que implica que la velocidad cruda se calcula a partir de diferencias de conteo en intervalos discretos, generando una señal escalonada con saltos de aproximadamente 50 pulsos/s debido a la resolución finita del *encoder*.

El valor de $\alpha = 0.3$ se seleccionó tras varias pruebas empíricas porque ofrece un equilibrio óptimo entre suavizado y retraso de fase. Con α más altos (p. ej., 0.5 o 0.7), el filtro sigue casi instantáneamente a la velocidad cruda, pero conserva gran parte del ruido de alta frecuencia, lo que provoca oscilaciones en la salida del controlador y un desgaste innecesario del motor. Con α más bajos (p. ej., 0.1), el suavizado es excelente, pero se introduce un retraso apreciable en la señal filtrada respecto a la real, lo que puede desestabilizar el lazo de control al retardar la realimentación.

Perfil trapezoidal con controlador de velocidad PID.

Una vez validado el controlador PID de velocidad en lazo cerrado, el siguiente paso fue incorporar un generador de trayectorias que proporcionara *setpoints* de velocidad suaves en el tiempo, en lugar de aplicar cambios bruscos (escalón) a la consigna. Esta estrategia es fundamental para reducir los picos de corriente, el desgaste mecánico y el deslizamiento de las ruedas, problemas identificados en la etapa de planteamiento del proyecto.

El generador implementado se basa en un perfil trapezoidal de velocidad, parametrizado por tres variables fundamentales:

- Distancia total a recorrer (X): definida en pulsos de encoder.
- Velocidad crucero máxima (V): establecida en pulsos por segundo.
- Aceleración máxima permitida (a): en pulsos por segundo al cuadrado.

A partir de estos parámetros, el generador calcula la evolución temporal de la velocidad de referencia, que luego se envía como *setpoint* dinámico al controlador PID. El siguiente extracto muestra las funciones implementadas para el cálculo de tiempos y la generación del perfil en tiempo real.

Código 3 – Generador de perfil trapezoidal de velocidad

```
“void trapezoidal_times(float dist, float vel, float acel) {  
    float min_distance = (vel * vel) / acel;  
  
    if (min_distance < fabsf(dist)) {                // TRAPECIO  
        triangle = false;  
        t_1 = vel / acel;  
        t_2 = fabsf(dist) / vel;  
        t_3 = t_1 + t_2;  
    } else {                                        // TRIÁNGULO  
        triangle = true;  
        peak_vel = sqrtf(fabsf(dist) * acel);  
        t_1 = peak_vel / acel;  
        t_2 = 2.0f * t_1;  
    }  
}
```

```
void IRAM_ATTR trapezoidal_profile() {
    now_count++;
    profile_time = (now_count - 1) * profile_DT;

    if (!triangle) {                                // TRAPÉCIO
        if (profile_time < t_1) {
            dynamic_setpoint = max_aceleration *
                profile_time;
        } else if (profile_time < t_2) {
            dynamic_setpoint = cruiser_vel;
        } else if (profile_time <= t_3) {
            dynamic_setpoint = cruiser_vel - max_aceleration
                * (profile_time - t_2);
        } else {
            dynamic_setpoint = 0.0f;
        }
    } else {                                        // TRIÁNGULO
        if (profile_time < t_1) {
            dynamic_setpoint = max_aceleration *
                profile_time;
        } else if (profile_time <= t_2) {
            dynamic_setpoint = peak_vel - max_aceleration *
                (profile_time - t_1);
        } else {
            dynamic_setpoint = 0.0f;
        }
    }
}
```

El código 3 es un fragmento el cual se implementa el generador de perfil trapezoidal mediante dos funciones complementarias. La función `trapezoidal_times()` se ejecuta una sola vez al inicio del movimiento (por ejemplo, en el `setup()`) o al recibir una nueva consigna de distancia), calculando los tiempos característicos t_1 , t_2 y t_3 (o t_2 para el caso triangular) en función de la distancia, la velocidad crucero y la aceleración máxima. Por su parte, la función `trapezoidal_profile()` se invoca periódicamente dentro de una rutina de interrupción temporizada con un período fijo `profile_DT` (en este caso, 20 ms), actualizando en cada iteración el *setpoint* dinámico de velocidad según la fase del movimiento en la que se encuentre el motor. Esta estructura permite desacoplar el cálculo paramétrico (que es computacionalmente ligero y se hace una sola vez) de la generación en tiempo real de la referencia, garantizando una ejecución eficiente y sincronizada con el lazo de control.

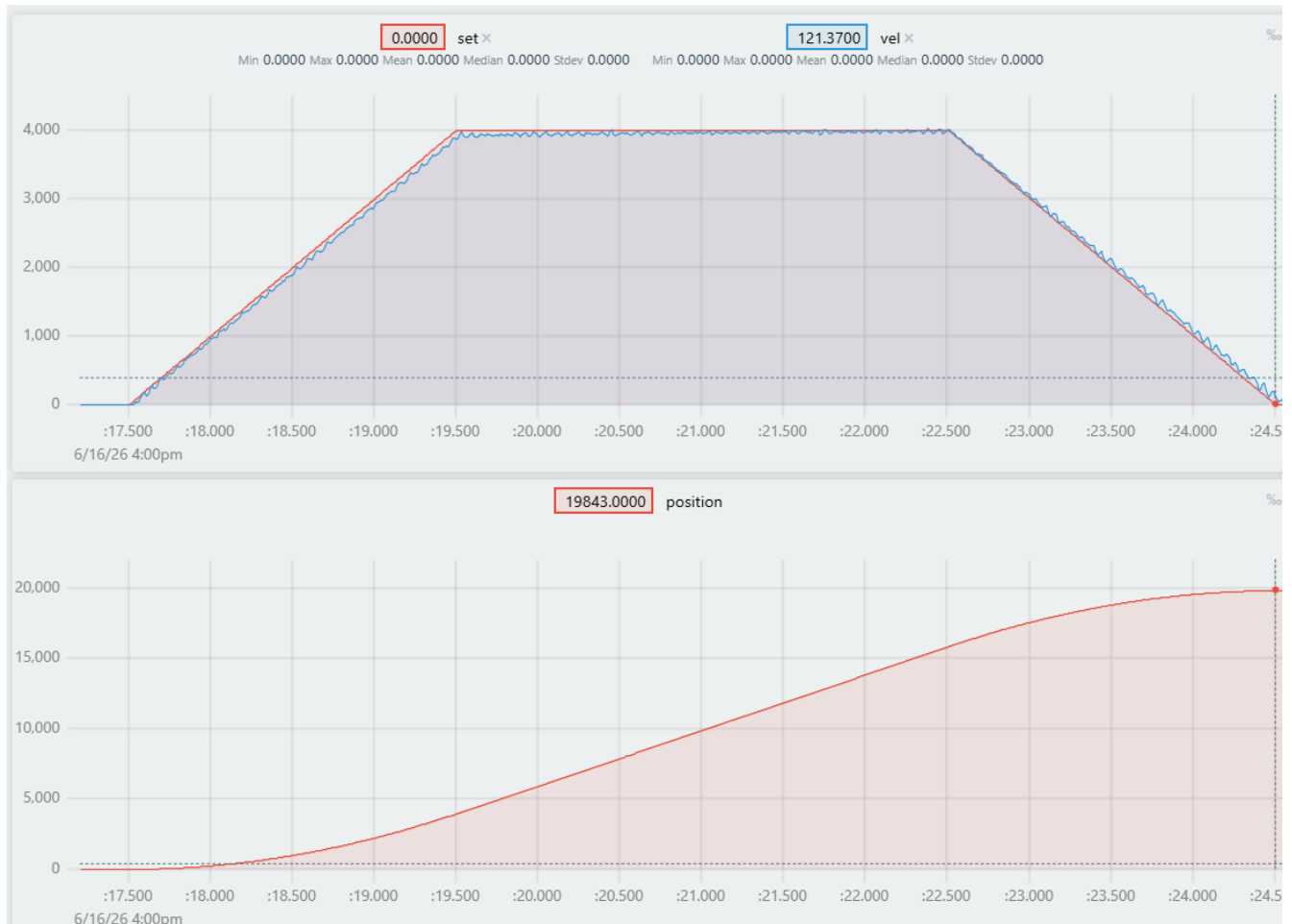


Figura 10 - Seguimiento del perfil trapezoidal de velocidad para distancia de 20,000 pulso, velocidad crucero de 4000 pps y aceleración de 2000 pps². Ganancias del controlador: $K_p = 15.3$, $K_i = 8.0$, $K_d = 0.083$.

La figura 10 muestra el comportamiento del sistema al aplicar un perfil trapezoidal de velocidad parametrizado con una distancia objetivo de 20,000 pulsos, una velocidad crucero de 4,000 pulsos/s y una aceleración de 2,000 pulsos/s². El controlador PID de velocidad emplea ganancias $K_p = 15.3$, $K_i = 8.0$ y $K_d = 0.083$, valores sintonizados para obtener una respuesta rápida y estable. En la gráfica se observa que la velocidad real (filtrada mediante EMA) sigue con fidelidad el *setpoint* generado, sin sobreimpulsos, lo que valida la correcta sintonía del PID y la efectividad del generador de trayectorias. Sin embargo, al finalizar el perfil —cuando el *setpoint* de velocidad retorna a cero— la posición alcanzada es de 19,843 pulsos, es decir, 157 pulsos por debajo de la consigna de 20,000. Este error residual es inherente al control de velocidad puro, ya que el lazo cerrado regula únicamente la velocidad y no la

posición; pequeñas discrepancias durante el seguimiento, junto con efectos de fricción y cuantificación del *encoder*, impiden que la integral del error de velocidad iguale exactamente la distancia programada.

Perfil trapezoidal con controlador de posición PD con doble retroalimentación.

Una vez validado el generador de perfil trapezoidal para velocidad, se extendió la estrategia para incluir el control de posición. En esta etapa, el generador de trayectorias no solo produce un *setpoint* de velocidad suave, sino que también calcula en tiempo real la posición deseada correspondiente a cada instante del movimiento, siguiendo la misma cinemática del perfil trapezoidal. De este modo, se dispone de dos referencias simultáneas: la velocidad y la posición.

Código 4 – Generador de perfil trapezoidal de velocidad y perfil S de posición.

```
void IRAM_ATTR trapezoidal_profile() {
    now_count++;
    profile_time = (now_count - 1) * profile_DT;

    if (!triangle) { // TRAPECIO
        if (profile_time < t_1) {
            vel_setpoint = max_acelaration * profile_time;
            pos_setpoint = max_acelaration/2.0f *
                (profile_time * profile_time);
        }
        else if (profile_time < t_2) {
            vel_setpoint = cruiser_vel;
            pos_setpoint = x_ac + cruiser_vel * (profile_time
                - t_1);
        }
        else if (profile_time <= t_3) {
            vel_setpoint = cruiser_vel - max_acelaration *
                (profile_time - t_2);
            pos_setpoint = (distance - x_ac) + cruiser_vel *
                (profile_time - t_2) - (max_acelaration/2.0f) *
                ((profile_time - t_2) * (profile_time - t_2));
        }
        else {
            vel_setpoint = 0.0f;
            pos_setpoint = distance;
        }
    }
}
```

```

    }
  }
  else { // TRIÁNGULO
    if (profile_time < t_1) {
      vel_setpoint = max_acelaration * profile_time;
      pos_setpoint = max_acelaration/2.0f *
        (profile_time * profile_time);
    }
    else if (profile_time < t_2) {
      vel_setpoint = peak_vel - max_acelaration *
        (profile_time - t_1);
      pos_setpoint = (distance - x_ac) + peak_vel *
        (profile_time - t_1) - (max_acelaration/2.0f) *
        ((profile_time - t_1) * (profile_time - t_1));
    }
    else {
      vel_setpoint = 0.0f;
    }
  }
}

void IRAM_ATTR PID_control() {
  c = encoder.getCount();

  raw_velocity = (float)(c - last_count) / DT;
  last_count = c;

  filtered_velocity = ALPHA * raw_velocity + (1.0f - ALPHA)
    * filtered_velocity;

  error_vel = vel_setpoint - filtered_velocity;
  error_pos = pos_setpoint - c;

  output = (KP * error_pos) + (KD * error_vel);

  int32_t abs_pwm = (int32_t)fabsf(output);
  if (abs_pwm < PWM_MIN) abs_pwm = PWM_MIN;
  if (abs_pwm > PWM_MAX) abs_pwm = PWM_MAX;

  if (pos_setpoint == distance && abs(error_pos) < 5) {
    ledcWrite(CH_LPWM, 0);
    ledcWrite(CH_RPWM, 0);
  }
  else {
    if (output > 0.0f) {
      ledcWrite(CH_RPWM, abs_pwm);
    }
  }
}

```

```

        ledcWrite(CH_LPWM, 0);
    }
    else if (output < 0.0f) {
        ledcWrite(CH_LPWM, abs_pwm);
        ledcWrite(CH_RPWM, 0);
    }
}
}
}

```

El fragmento de código muestra la implementación del generador de trayectorias y el controlador de doble retroalimentación, materializando la estrategia descrita en la metodología. Se emplean dos rutinas de interrupción con período fijo de 10 ms cada una: la primera, *trapezoidal_profile()*, actualiza en cada iteración los *setpoints* de velocidad y posición según las leyes horarias del perfil trapezoidal (determinando previamente si el perfil es trapezoidal o triangular mediante la función *trapezoidal_times()*); la segunda, *PID_control()*, lee el *encoder*, filtra la velocidad mediante EMA y calcula la señal de control como $\text{output} = K_P * \text{error_pos} + K_D * \text{error_vel}$, combinando así ambos errores en una arquitectura de doble retroalimentación. Después de la saturación se programo una zona muerta de ± 5 pulsos.



Figura 11 -Seguimiento del perfil parabólico – lineal – parabólico de posición para distancia de 20,000 pulso, velocidad crucero de 4000 pps y aceleración de 2000 pps². Ganancias del controlador: $K_p = 64$, $K_d = 2.33$.

En la figura 11 se observa como en el instante en que la referencia de posición es de 20000 pulsos el perfil de posición lleva un acumulado de 19997 y es lo esperado debido a que se programó una zona muerta de ± 5 pulsos.

Sincronización de perfiles

La sincronización de perfiles consiste en escalar proporcionalmente la velocidad cruceo y la aceleración máxima del motor que tenga que recorrer menor distancia punto a punto. Ambos generadores de trayectoria tienen como constantes las misma velocidades y aceleraciones, aunque se pueden configurar según la necesidad, y solo aquel que tenga que recorrer mas distancia conservara sus parámetros iniciales.

Código 5 – Sincronización de perfiles.

```
“void trapezoidal_times_m1(float escale_factor, float vel,
float acel) {

    if (distance_m1 < distance_m2){
        escale_factor = distance_m1 / distance_m2;
        vel = escale_factor * vel;
        acel = escale_factor * acel;
    }

    float min_distance = (vel * vel) / acel;
    x_ac_m1 = (vel * vel) / (2.0f * acel);

    if (min_distance < fabsf(distance_m1)) {    // TRAPECIO
        triangle_m1 = false;
        t_1_m1 = vel / acel;
        t_2_m1 = fabsf(distance_m1) / vel;
        t_3_m1 = t_1_m1 + t_2_m1;
    }
    else {    // TRIÁNGULO
        triangle_m1 = true;
        peak_vel_m1 = sqrtf(fabsf(distance_m1) * acel);
        t_1_m1 = peak_vel_m1 / acel;
        t_2_m1 = 2.0f * t_1_m1;
    }
}

//
void trapezoidal_times_m2(float escale_factor, float vel,
float acel) {
```

```
if (distance_m2 < distance_m1){
    escale_factor = distance_m2 / distance_m1;
    vel = escale_factor * vel;
    acel = escale_factor * acel;
}

float min_distance = (vel * vel) / acel;
x_ac_m2 = (vel * vel) / (2.0f * acel);

if (min_distance < fabsf(distance_m2)) {    // TRAPECIO
    triangle_m2 = false;
    t_1_m2 = vel / acel;
    t_2_m2 = fabsf(distance_m2) / vel;
    t_3_m2 = t_1_m2 + t_2_m2;
}
else {                                     // TRIÁNGULO
    triangle_m2 = true;
    peak_vel_m2 = sqrtf(fabsf(distance_m2) * acel);
    t_1_m2 = peak_vel_m2 / acel;
    t_2_m2 = 2.0f * t_1_m2;
}
}
```

En este fragmento se puede ver como el escalamiento se hace en las funciones llamadas *trapezoidal_times* las cuales se ejecutan antes de generar los perfiles, estas funciones definen los tiempos de transición entre las fases de las trayectorias. En las primeras líneas de las funciones se aprecia una condición cuya finalidad es saber si se tienen que recorrer diferentes distancias y definir que motor será el maestro (el de mayor distancia). Es importante mencionar que esta función no modifica las variables globales *cruiser_vel* y *max_acelARATION* las cuales parametrizan los generadores de trayectoria, sino que trabajan con variables locales y con ello desencadenan la modificación de los tiempos de transición de ser necesario.

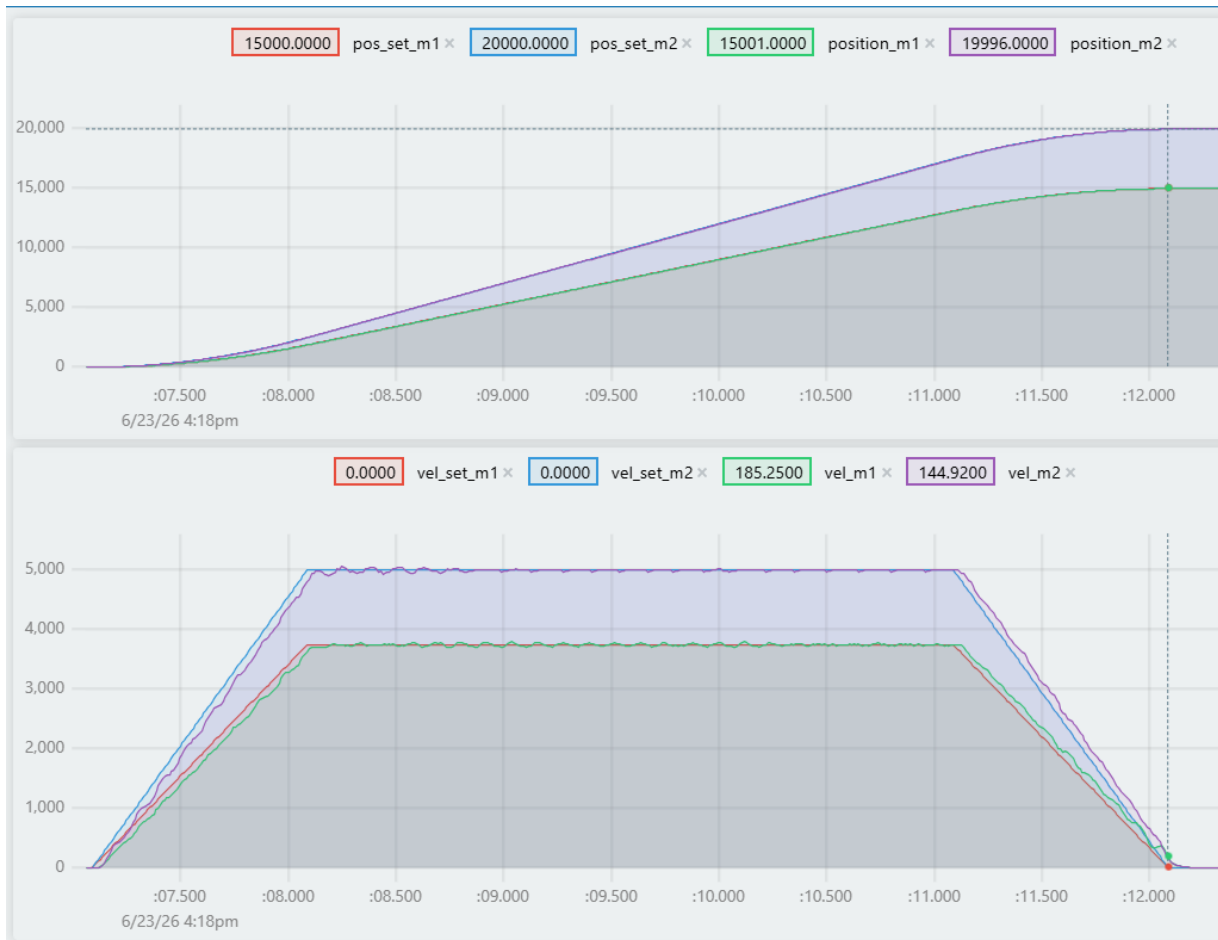


Figura 11 -Sincronización de dos perfiles parabólicos -lineales -parabólicos de posición para su seguimiento con dos controladores PD de posición con doble retroalimentación para cada motor, con 20000 y 15000 pulsos como referencia de posición respectivamente, velocidad crucero de 4000 pps y aceleración de 2000 pps².

En la figura 11 se aprecia como ambas trayectorias comparten los mismos tiempos de transición para sus fases de aceleración y desaceleración. Sus constantes de ganancia K_p es de 100 y sus K_d es de 1.5, cabe mencionar que para estos resultados se utilizó el *driver* para motores TB6612FNG ya que cuenta con dos canales y es más eficiente que el tradicional L298N. Tengo que reconocer que me resulto complicado suavizar la velocidad sin que se retrasara con su consigna, pero lo esperado es un control de posición satisfactorio. En el instante en el que la consigna para el motor 2 (el maestro) fue de 20000 pulsos su posición real fue 19996, al estar fuera de la zona muerta en el siguiente instante se aproximó más a la referencia. El motor 1 (el esclavo) se pasó por 1 pulso el cual es completamente aceptable y esta dentro de la zona muerta.

Etapas final:

El logro de los resultados experimentales requirió la integración y operación conjunta de la fase 3 y 4, teniendo como punto de partida el montaje y ensamblaje del chasis y sus componentes.

Armado del robot diferencial

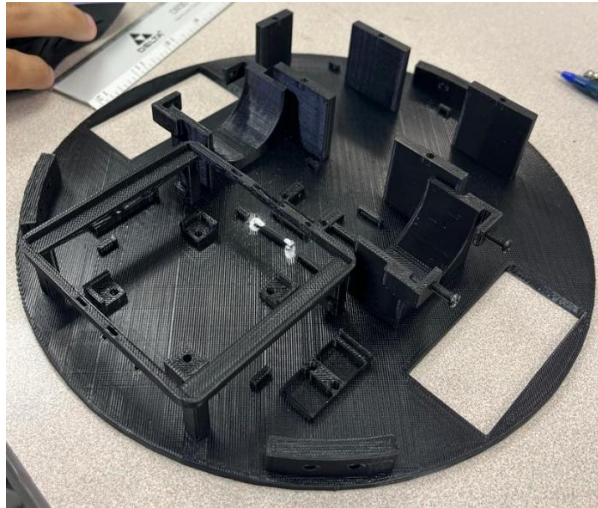


Figura 12 – Base principal impresa con PETG

En la figura 12 se muestra la base del robot diferencial la cual presenta un diámetro de 21.5 cm y tiene un grosor de 3 mm, su de impresión fue de aproximadamente de 6 horas y salió en buenas condiciones. Se contemplaron tolerancias de 0.4 mm para montar los componentes sin forzar mecánicamente sus cajones, bases o perforaciones.

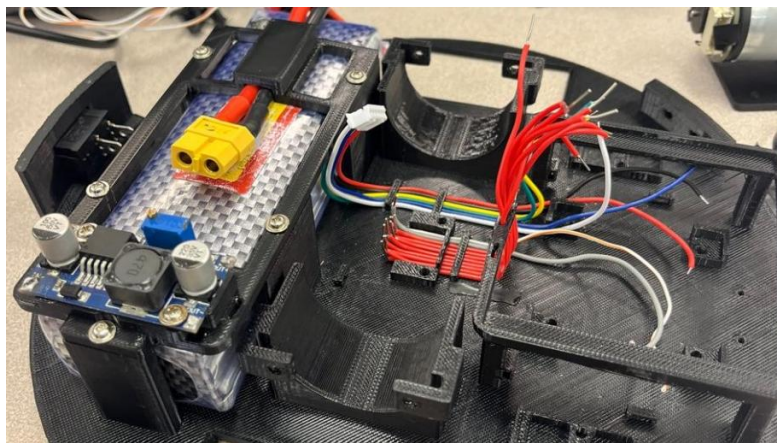


Figura 12.1 – Montaje de la batería con su tapa y gestión de cables para la IMU y motores.

En la figura 12.1 se aprecia el uso de las guías para el cableado cuyo diseño se basó en los clips para cables comerciales. Todos los cables son guiados hacia la parte frontal del robot en la cual esta la zona de control y potencia.

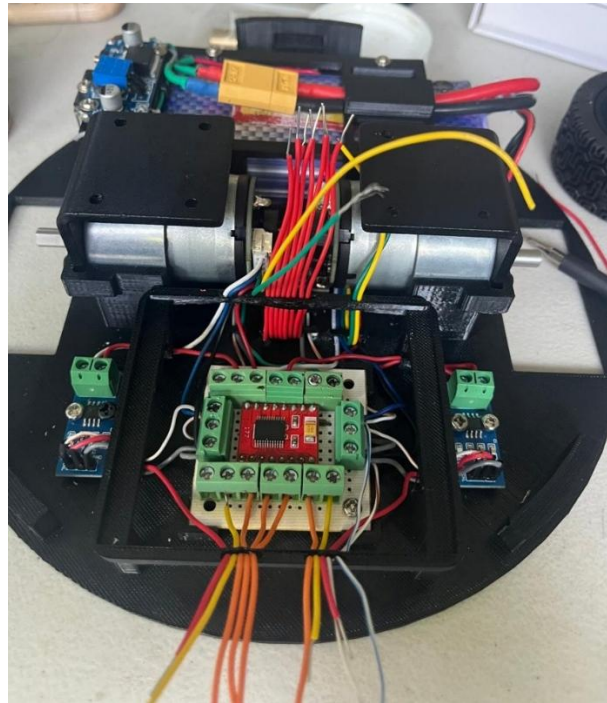


Figura 12.2 – Gestión completa del cableado de los componentes hacia la placa del *driver* para motores donde se centraliza la alimentación y control.

En la figura 12.2 se aprecia la placa perforada con terminales que se ensambla para el *driver* y para la alimentación de 5V para todos los sensores de orientación, de corriente y para el microcontrolador. Se muestra como es la parte inferior de la base que suspende la ESP32 la cual cuenta con surcos para la distribución de cables hacia los pines del microcontrolador.

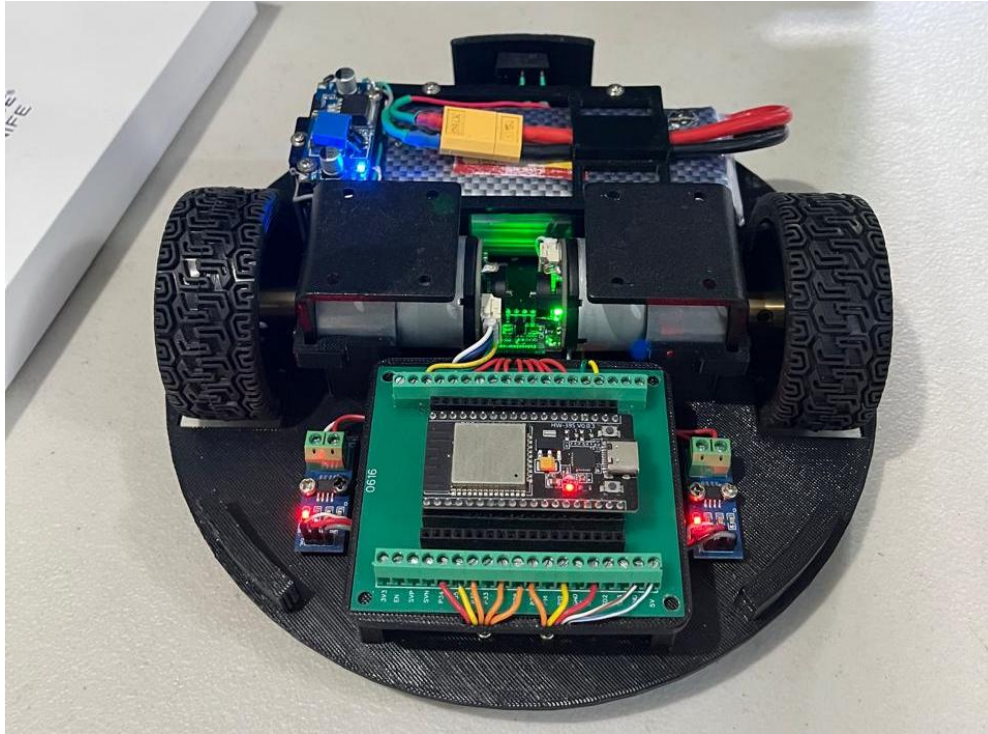


Figura 12.3 – Ensamble del robot diferencial completamente funcional.

En la figura 12.3 se muestra el robot diferencial con todos los componentes eléctricos y mecánicos montados en el chasis. En la parte trasera se encuentra un switch para encender el robot y finalmente operarlo en su forma sin cobertura superior.

Robot diferencial con perfil trapezoidal de velocidad y telemetría.

Se implementaron los perfiles trapezoidales de velocidad en ambos motores del robot con la finalidad de desplazarlo linealmente una determinada distancia en metros siguiendo la trayectoria generada con controladores PD de posición. Se empleó un *WebSocket* para la telemetría y mandar así desde un html en un dispositivo conectado por Wifi una distancia a seguir configurando la velocidad crucero y aceleración máxima.

Código 6 – Rutina de interrupción según el estado de la *WebSocket*.

```
“void onWsEvent(AsyncWebSocket *server, AsyncWebsocketClient
    *client,AwsEventType type, void *arg, uint8_t *data, size_t
    len) {
```

```
else if (type == WS_EVT_DATA) {
    String msg = String((char*)data).substring(0, len);

    if (msg == "TOGGLE") {
        trayectoria_activa = !trayectoria_activa;
        ws.textAll(trayectoria_activa ? "ESTADO:1" :
            "ESTADO:0");
        ws.textAll(trayectoria_activa ? "EVENTO:Trayectoria
            iniciada" : "EVENTO:Trayectoria detenida");
        Serial.println(trayectoria_activa ?
            "TRAYECTORIA_INICIADA" : "TRAYECTORIA_DETENIDA");
        return;
    }

    int sep = msg.indexOf(':');
    if (sep == -1) return;

    String clave = msg.substring(0, sep);
    String valor = msg.substring(sep + 1);

    if (clave == "KP1") {
        KP_m1 = valor.toFloat();
        ws.textAll("PID_M1:" + String(KP_m1) + "," +
            String(KD_m1));
    }
    else if (clave == "KD1") {
        KD_m1 = valor.toFloat();
        ws.textAll("PID_M1:" + String(KP_m1) + "," +
            String(KD_m1));
    }
    else if (clave == "KP2") {
        KP_m2 = valor.toFloat();
        ws.textAll("PID_M2:" + String(KP_m2) + "," +
            String(KD_m2));
    }
    else if (clave == "KD2") {
        KD_m2 = valor.toFloat();
        ws.textAll("PID_M2:" + String(KP_m2) + "," +
            String(KD_m2));
    }
    else if (clave == "DIST") {
        float metros = valor.toFloat();
        distance_m1 = metros * PULSES_PER_METER;
        distance_m2 = metros * PULSES_PER_METER;
        ws.textAll("PARAMS:" + String(metros) + ",")
    }
}
```


- **Conversión y asignación:** Evalúa las claves: KP1, KD1, KP2, KD2, DIST, VELCRU, ACEL, y convierte el texto del valor a formato numérico de punto flotante usando `toFloat()`.
- **Transformación de unidades:** Para las variables de movimiento, realiza la conversión dimensional multiplicando el valor en metros, m/s o m/s² por la constante `PULSES_PER_METER` para convertir las unidades de ingeniería a cuentas discretas de *encoder*.
- **Bucle de retroalimentación:** Inmediatamente después de actualizar las variables globales en memoria RAM, el código construye un paquete de respuesta estructurado y lo transmite a todos los clientes mediante `ws.textAll()`, garantizando la sincronización de la interfaz gráfica sin necesidad de peticiones de refresco (*polling*).

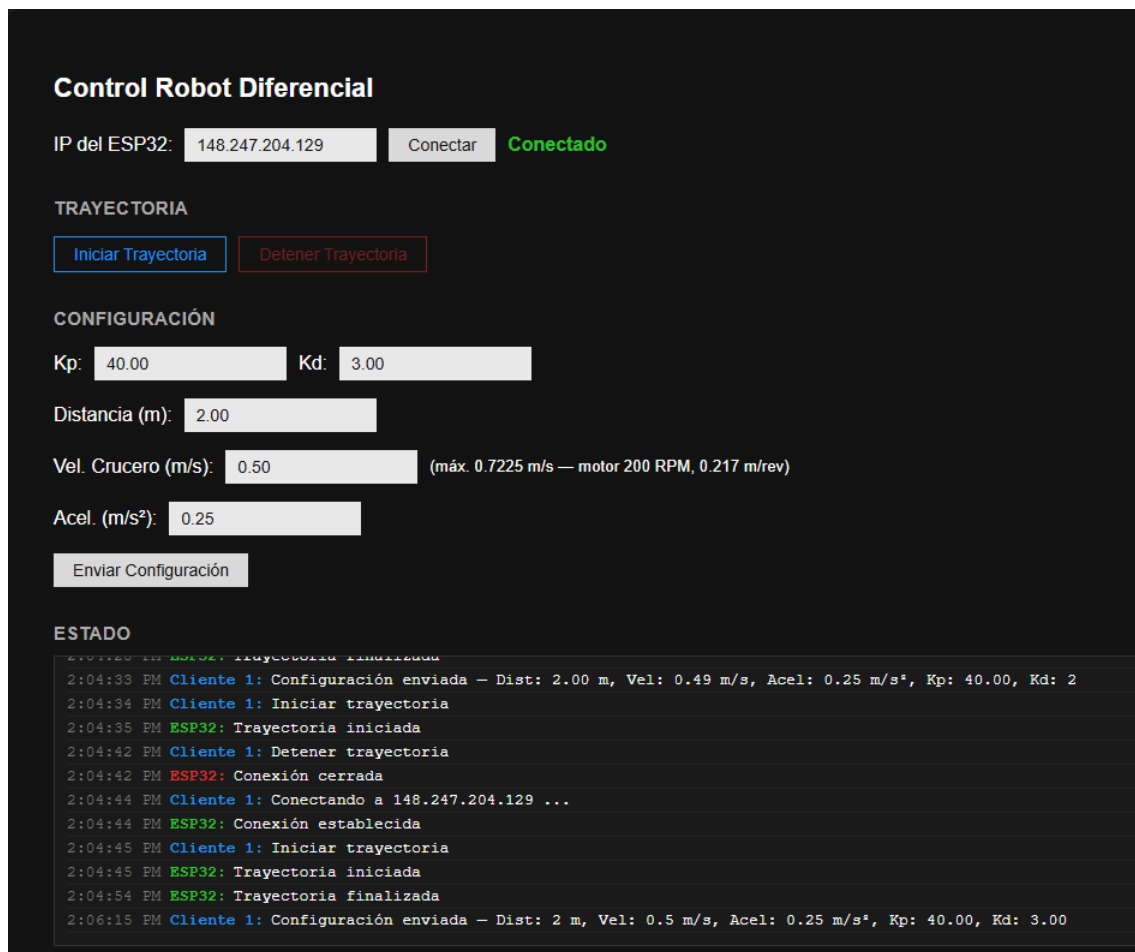


Figura 13 – Interfaz web de control del robot diferencial vía WebSocket, mostrando el panel de configuración y el log de estado en tiempo real.

La Figura 13 muestra el panel de control web desarrollado para operar el robot de forma remota a través de WebSocket. En la parte superior se ingresa la dirección IP asignada al ESP32, la cual se obtiene previamente por comunicación serial durante el arranque del microcontrolador (impresa una vez que este se conecta a la red WiFi), y se establece la conexión mediante el botón "Conectar". Una vez establecido el enlace, el panel de "Configuración" permite definir los parámetros de operación de la trayectoria, ganancias proporcional y derivativa del controlador PD, distancia a recorrer en metros, velocidad cruce y aceleración máxima, los cuales se transmiten al ESP32 en conjunto mediante el botón "Enviar Configuración". La sección "Trayectoria" permite iniciar o detener la ejecución del perfil trapezoidal de velocidad de forma manual. Finalmente, el panel "Estado" funciona como bitácora de eventos: despliega en tiempo real las confirmaciones enviadas por el ESP32 (recepción de parámetros, inicio y finalización de la trayectoria) junto con las acciones generadas desde el cliente, cada una marcada con su hora y origen (Cliente o ESP32), lo que permite verificar que la comunicación bidireccional entre la interfaz y el firmware se realiza correctamente.

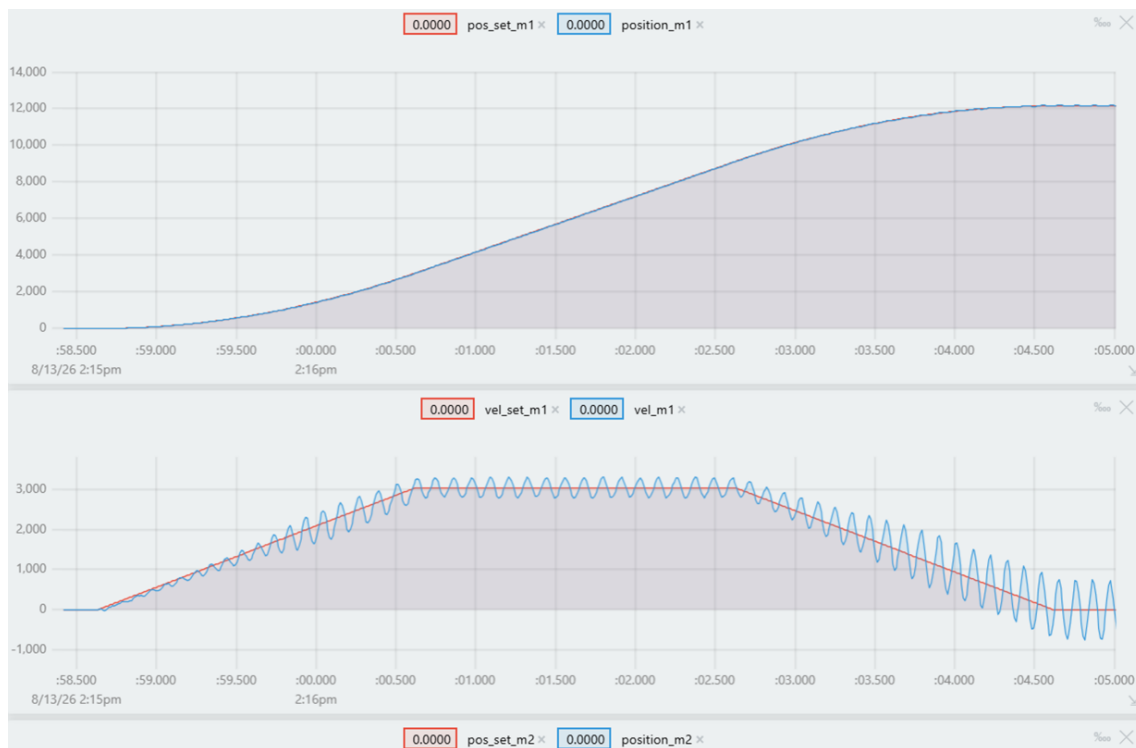


Figura 13.1 – Seguimiento de un perfil parabólico - lineal - parabólico de posición con un controlador P con 180 de ganancia proporcional. Parametrizado a 2 metros con una velocidad de 0.5 m/s y una aceleración de 0.25 m/s^2 .

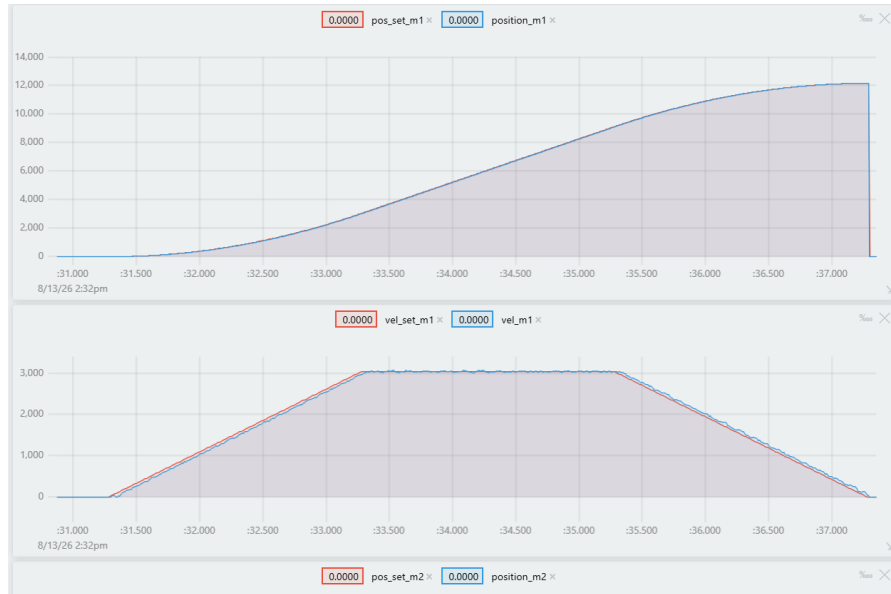


Figura 13.2 – Seguimiento de un perfil parabólico - lineal - parabólico de posición con un controlador PD con 100 de ganancia proporcional y 0.2 de ganancia derivativa. Parametrizado a 2 metros con una velocidad de 0.5 m/s y una aceleración de 0.25 m/s^2 .

En la figura 13.2 se muestra la mejor sintonización de las ganancias del controlador para el seguimiento de la posición aplicada en ambos motores para mover al robot diferencial. Se aplicó un perfil y un controlador para cada motor.



Figura 13.2 – Seguimiento de un perfil parabólico - lineal - parabólico de posición con un controlador PD con 100 de ganancia proporcional y 0.2 de ganancia derivativa. Parametrizado a 1 metros con una velocidad cruce de 0.65 m/s y una aceleración de 0.35 m/s^2 .

En la figura 13.2 se muestra el caso donde el perfil no alcanza la velocidad crucero por lo que empieza a desacelerar según los tiempos de transición para cumplir con la distancia y las rampas de aceleración parametrizadas

Pruebas del desplazamiento lineal del robot diferencial con perfil trapezoidal de velocidad.



Figura 14 – Línea de partida para las pruebas de desplazamiento utilizando una cinta métrica.

La figura 14 muestra la zona de pruebas en la que controle mi robot diferencial con un generador de trayectoria punto a punto el cual configure en todas pruebas una velocidad crucero de 0.5 m/s y una aceleración de $0.15/\text{s}^2$. A continuación, se mostrarán los resultados según las distancias parametrizadas:

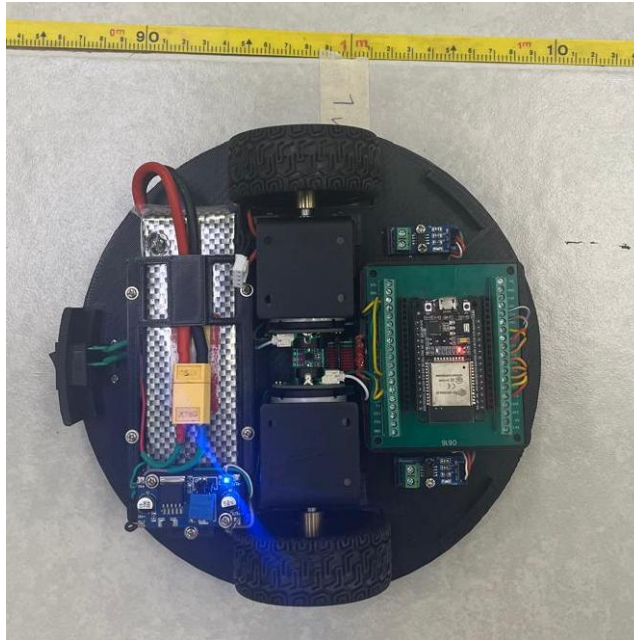


Figura 14.1 – Resultados del desplazamiento del robot parametrizado a recorrer 1 metro de distancia.

En la figura 14.1 se ven los resultados al configurar el perfil trapezoidal de velocidad a un metro de distancia. El robot se desplazó aproximadamente 98 cm.

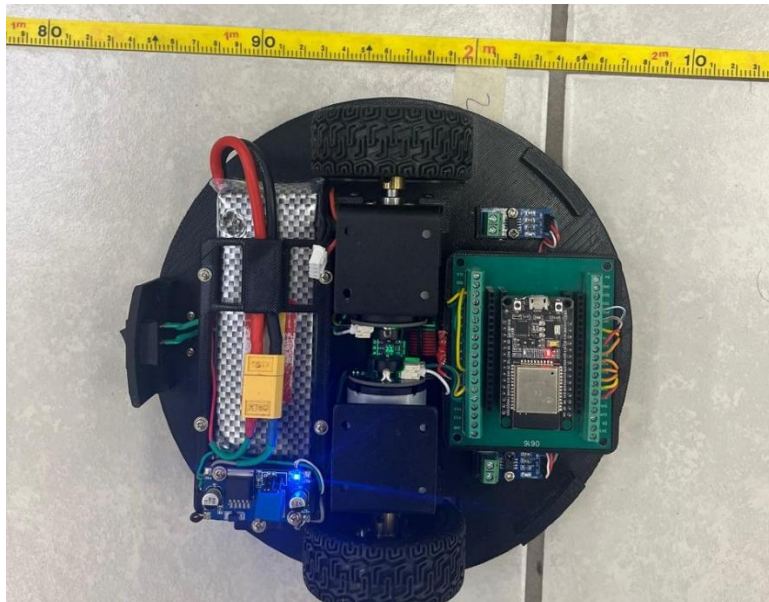


Figura 14.2 – Resultados del desplazamiento del robot parametrizado a recorrer 2 metro de distancia.

En la figura 14.2 se ven los resultados al configurar el perfil trapezoidal de velocidad a dos metros de distancia. El robot se desplazó aproximadamente 196 cm.

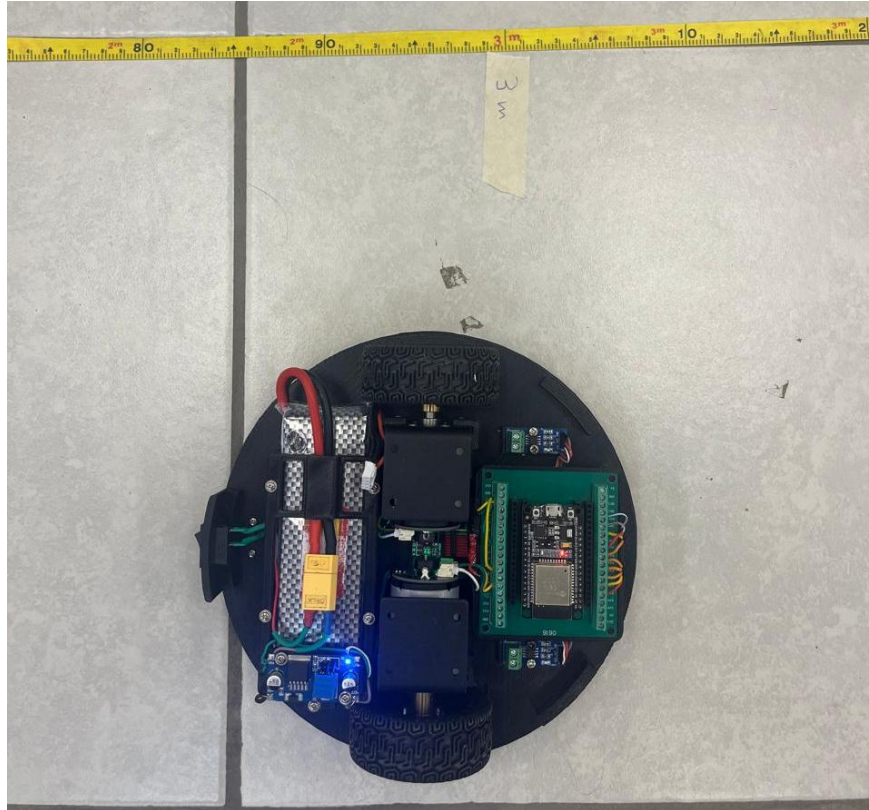


Figura 14.3 – Resultados del desplazamiento del robot parametrizado a recorrer 3 metro de distancia.

En la Figura 14.3 se presentan los resultados obtenidos al configurar el perfil trapezoidal de velocidad para un desplazamiento objetivo de tres metros. El robot registró un recorrido real de aproximadamente 295 cm, evidenciando una desviación en su orientación a partir del segundo metro de trayectoria. De estos datos se concluye que el sistema acumula un desfase lineal aproximado de 2 cm por cada metro recorrido. Esta discrepancia se atribuye principalmente a imprecisiones en la determinación del radio efectivo de las ruedas, así como a la presencia de holguras o juegos mecánicos en los acoples de transmisión.

Conclusiones.

Resultados y actividades relevantes

Se diseñó, construyó y validó un robot móvil de tracción diferencial integrando un generador de perfil trapezoidal de velocidad y un algoritmo PD de doble retroalimentación, deslizamiento de las ruedas sobre la superficie de rodamiento, logrando un seguimiento de trayectoria suave, sobreimpulso nulo y detención precisa. En las pruebas físicas sobre el chasis de *PETG*, la plataforma alcanzó una elevada repetibilidad en desplazamientos lineales de 1 m, 2 m y 3 m, registrando distancias reales de 98 cm, 196 cm y 295 cm respectivamente, lo que determinó un desfase lineal sistemático de 2 cm por metro recorrido, monitoreado en tiempo real mediante telemetría asíncrona *WebSockets*.

Experiencia personal adquirida

La estancia dentro del Laboratorio de Robótica de CINVESTAV Unidad Tamaulipas fortaleció la resiliencia y la capacidad de análisis crítico para la resolución autónoma de problemas técnicos en hardware y software, permitiendo además desarrollar disciplina laboral, gestión eficiente del tiempo bajo cronograma y habilidades de colaboración proactiva en un entorno de investigación de alto nivel.

Experiencia profesional adquirida

Se logró un dominio integral del ciclo de desarrollo mecatrónico, consolidando competencias en el diseño estructurado en SolidWorks, manufactura aditiva en PETG, instrumentación electrónica de potencia, modelado cinemático diferencial, programación embebida en C/C++ sobre el microcontrolador ESP32 mediante PlatformIO y creación de servidores web interactivos.

Sugerencias de mejora al Programa Académico

Integrar en el plan de estudios el desarrollo de interfaces web y la aplicación de protocolos de baja latencia (*WebSockets*, *MQTT*) orientados a la supervisión inalámbrica de plataformas móviles.

Referencias.

- [1] Espressif Systems, "ESP32 Technical Reference Manual," ver. 5.8, Espressif Systems, Shanghai, China, 2023. [En línea]. Disponible en: <https://bit.ly/45tJxvn>
- [2] Infineon Technologies, "BTS 7960 High Current PN Half Bridge NovalithIC™," Data Sheet, Rev. 1.1, dic. 2004.
- [3] Urany, "¿Encoder Incremental o Absoluto? Características y Aplicaciones," Urany Blog, 7 de noviembre de 2024. [En línea]. Disponible en: <https://urany.net/blog/encoder-incremental-o-absoluto> [Accedido: 20 -mayo-2026].
- [4] T. Garrett, "Salida de Encoder en Cuadratura: Principios Técnicos y Aplicaciones," Industrial Monitor Direct, 28 de abril de 2026. [En línea]. Disponible en: <https://industrialmonitordirect.com/es/blogs/knowledgebase/quadrature-encoder-output-technical-principles-and-applications>. [Accedido: 25-mayo-2026].
- [5] WPILib Developers, "Introducción a PID," WPILib Documentation, FIRST Robotics Competition, 2023. [En línea]. Disponible en: <https://docs.wpilib.org/es/2023/docs/software/advanced-controls/introduction/introduction-to-pid.html>. [Accedido: 3- jul-2026].
- [6] K. Ogata, Modern Control Engineering, 5.ª ed. Upper Saddle River, NJ, EE. UU.: Prentice Hall, 2010.
- [7] Ardumania, "Controlador PID: Qué es, cómo funciona y aplicaciones en la automatización actual," Ardumania, 2 de diciembre de 2025. [En línea]. Disponible en: <https://ardumania.es/que-es-el-controlador-pid/> [Accedido: 4-jul-2026].
- [8] G. Hunter, "Exponential Moving Average (EMA) Filters," mbedded.ninja, 31 de mayo de 2026. [En línea]. Disponible en: <https://blog.mbedded.ninja/programming/signal-processing/digital-filters/exponential-moving-average-ema-filter/>. [Accedido: 10-jul-2026].
- [9] C. Lewin, "Mathematics of Motion Control Profiles," Performance Motion Devices, s.f. [En línea]. Disponible en: https://www.pmdcorp.com/math_of_mc_profiles.pdf. [Accedido: 10-jul-2026].
- [10] H. Chen, H. Mu, y Y. Zhu, "Real-time Generation of Trapezoidal Velocity Profile for Energy Saving in Servomotor Systems," J. Mech. Eng., vol. 54, no. 7, pp. 153-160, abr. 2018. doi: 10.3901/JME.2018.07.152.
- [11] L. Biagiotti y C. Melchiorri, Trajectory Planning for Automatic Machines and Robots. Berlín, Alemania: Springer-Verlag, 2008.

- [12] ESP32 I/O, "ESP32 - WebSocket," esp32io.com, s.f. [En línea]. Disponible en: <https://esp32io.com/tutorials/esp32-websocket>. [Accedido: 8-ago-2026].

Anexos.

Seguro facultativo.

Instituto Mexicano del Seguro Social		
Constancia de Vigencia de Derechos		
Homoclave del trámite	Homoclave del formato	Fecha de publicación del formato en el DOF
IMSS-02-020	FF-IMSS-012	10 / 11 / 2015 DD MM AAAA
Datos Generales		
	NSS:	02240625927
	CURP:	AUMI061217HTSBRSA2
	Nombre(s), primer apellido y segundo apellido:	ISMAEL ISAAC ABUNDIS MARTINEZ
	Sexo:	Hombre
	Fecha de nacimiento:	17/12/2006
	Lugar de nacimiento:	TAMAULIPAS
Datos de Aseguramiento		
Con derecho al servicio médico:	SI	
Vigente:	31/03/2026	
Delegación:	TAMAULIPAS	
UMF:	UMF 067 SAN LUISITO	
Turno:	VESPERTINO	
Consultorio:	CONSULTORIO 5	
Agregado Médico:	1M2006ES	
Datos de Aseguramiento		
Registro Patronal	Nombre o razón social	
F0543370327	UNIVERSIDAD POLITÉCNICA DE VICTORIA	
Modalidad de Aseguramiento	Descripción de Modalidad	
MODALIDAD 32	SEGURO FACULTATIVO ESTUDIANTES	
Detalle de vigencia		
Estado	Inicio de Vigencia	Fecha de Constancia
VIGENTE	04/11/2025	31/03/2026
Beneficiarios		
NO APLICA		

De conformidad con los artículos 4 y 69-M, fracción V de la Ley Federal de Procedimiento Administrativo, los formatos para solicitar trámites y servicios deberán publicarse en el Diario Oficial de la Federación (DOF)

Carta de presentación.



Ciudad Victoria,
Tamaulipas, a 01
de Mayo
de 2026

**CENTRO DE INVESTIGACIÓN Y ESTUDIOS AVANZADOS DEL INSTITUTO
POLITÉCNICO NACIONAL (CINVESTAV TAMAULIPAS)
JOSÉ GABRIEL RAMÍREZ TORRES
PRESENTE**

La Universidad Politécnica de Victoria tiene a bien presentar a **ABUNDIS MARTINEZ ISMAEL ISAAC** estudiante del programa académico de **INGENIERÍA EN MECATRÓNICA** en su modalidad de **AUTOMATIZACIÓN**, con número de matrícula **2430256** y seguro facultativo IMSS número **02240625927**; quien deberá realizar su práctica profesional de **ESTADÍA TSU**, a partir del 04 de Mayo de 2026 al 14 de Agosto de 2026, con duración de **600 horas** desarrollando el proyecto **"ROBOT DIFERENCIAL CON CONTROL DE TRAYECTORIA MEDIANTE PERFIL TRAPEZOIDAL Y PID"** que le fue asignado.

Al concluir se le extenderá la carta de liberación al evaluarlo satisfactoriamente y además le solicitaremos su valiosa opinión respondiendo el formulario que se le enviará por correo electrónico, referente al desempeño del practicante, la información que nos proporcione es de vital importancia para la mejora de los programas académicos que ofrece la Universidad Politécnica de Victoria para la formación de profesionistas altamente especializados.

El practicante deberá cumplir con el Reglamento Interno aplicable al personal en su centro de trabajo.

Sin otro particular.

ATENTAMENTE

**OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN**

UNIVERSIDAD POLITÉCNICA DE VICTORIA

Av. Nuevas Tecnologías 5902
Parque Científico y Tecnológico de Tamaulipas
Carretera Victoria-Soto La Marina Km. 5.5
Cd. Victoria, Tamaulipas. C.P. 87138

Tel: (834) 1711100 al 10
www.upvictoria.edu.mx



Carta de aceptación.



"2026, Año de Margarita Maza Parada"

Cd. Victoria, Tamaulipas, a 6 de mayo del 2026.
UT/CA/060526/076
Asunto: Carta de aceptación a Estadía.

M. A. Othon Cano Garza
Director de Vinculación
Universidad Politécnica de Victoria
Presente,

Por este medio le comunico que el alumno Ismael Isaac Abundis Martínez, de la carrera de Ingeniería en Mecatrónica de la Universidad Politécnica de Victoria con número de control 2430256, ha sido ACEPTADO para realizar su Estadía TSU en el Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional (Cinvestav) Unidad Tamaulipas con sede en Ciudad Victoria; comprendido del 4 de mayo al 14 de agosto 2026.

El C. Ismael Isaac Abundis Martínez estará vinculado directamente a las actividades del proyecto de investigación titulado "Robot diferencial con control de trayectoria mediante perfil trapezoidal y pid", bajo mi dirección.

Al concluir las actividades de estadía y a petición del profesor anfitrión, el Cinvestav Unidad Tamaulipas entregará la carta de liberación de Estadía TSU del alumno únicamente a través del mecanismo institucional de la Universidad Politécnica de Victoria que tenga a bien indicarnos, por ejemplo: departamento de servicios escolares, departamento de vinculación, responsable del programa académico.

Sin otro particular, me despido de usted enviándole un cordial saludo.

Ateentamente

Dr. José Gabriel Ramírez Torres
Coordinador Académico de la Unidad Tamaulipas

Carretera Victoria-Soto la Marina, Km. 5.5.
Ciudad Victoria C.P. 87130
Ciudad Victoria Tamaulipas, México.
Tel: 834 107 0235 www.cinvestav.mx/tamaulipas

Carta de liberación.



"2026, Año de Margarita Maza Parada"

Cd. Victoria, Tamaulipas, a 12 de agosto del 2026.
UT/CA/120826/123
Asunto: Carta liberación.

M. A. Othon Cano Garza
Director de Vinculación
Universidad Politécnica de Victoria
Presente,

Por este medio se hace constar que el C. Ismael Isaac Abundis Martínez, estudiante de Ingeniería en Mecatrónica de la Universidad Politécnica de Victoria, con número de control 2430256; ha concluido satisfactoriamente su periodo de práctica profesional de ESTADÍA TSU en la Unidad Tamaulipas del Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional (Cinvestav) con sede en Ciudad Victoria, bajo mi dirección.

El C. Ismael Isaac Abundis Martínez ha desarrollado satisfactoriamente actividades del proyecto de investigación *"Robot diferencial con control de trayectoria mediante perfil trapezoidal"* durante el periodo comprendido del 4 de mayo al 14 de agosto del año en curso, con una duración total de 600 horas.

Se extiende la presente constancia en Ciudad Victoria, Tamaulipas; a doce días del mes de agosto del año dos mil veintiséis.

Atentamente


Dr. José Gabriel Ramírez Torres
Coordinador Académico de la Unidad Tamaulipas

RÚBRICA PARA LA EVALUACIÓN DEL REPORTE DE ESTADÍA

NOMBRE DEL ESTUDIANTE: ISMAEL ISAAC ABUNDIS MARTÍNEZ

PERIODO: 4 MAYO AL 14 AGOSTO 2026

NOMBRE DEL EVALUADOR: DR. MANUEL BENJAMÍN ORTIZ MOCTEZUMA

FIRMA DEL EVALUADOR:

CALIFICACIÓN FINAL: 96 /100

VALOR	CARACTERÍSTICAS A CUMPLIR	PUNTOS OBTENIDOS	OBSERVACIONES
6	Resumen ejecutivo y abstract: Indica qué se realizó, cómo se realizó y qué se obtuvo. (una cuartilla en español e inglés)	6	
6	Contenido temático: Describe correctamente el contenido del documento	6	
6	Introducción: El informe cumple con introducir al lector de una manera atractiva el panorama general del trabajo realizado en la unidad económica.	6	
6	Descripción de la unidad económica: Se redactan los detalles de los antecedentes del departamento, la empresa, giro, misión, visión, infraestructura, ubicación.	6	
6	Objetivos operativos: Describen las acciones a cumplir mediante actividades establecidas por el asesor empresarial	6	
24	Metodología: Descripción a detalle de las etapas y procedimientos utilizados en el proyecto	22	
18	Resultados: Interpreta y analiza los resultados obtenidos durante su estadía.	16	
12	Conclusiones: Las conclusiones son claras y acordes con el objetivo esperado	12	
6	Bibliografía y anexos: Cita correctamente las fuentes de información utilizando un estilo de referencia; anexa cartas correspondientes	6	
10	Responsabilidad: Entregó el reporte en la fecha y hora señalada. Asistió al menos al 80% de sus días de Estadía	10	

RÚBRICA PARA EXPOSICIÓN DEL REPORTE DE ESTADÍAS

Técnico Superior Universitario

Criterio	Nivel de logro		
	Bueno (10%)	Regular (7%)	Menor a lo esperado (5%)
Dominio del tema (10%)	Utiliza todos los términos y conceptos de manera correcta.	Utiliza la mayoría términos y conceptos de manera correcta	Confunde los términos y conceptos.
Presentación de material visual (10%)	*Las diapositivas usadas tienen fondo claro, letras, tablas e imágenes visibles. *La presentación contiene los aspectos relevantes de la Estadía	*Las diapositivas usadas presentan dificultad para su apreciación visual. *La presentación contiene los aspectos relevantes de la Estadía	*Las diapositivas usadas presentan dificultad para su apreciación visual. *La presentación carece de algunos aspectos relevantes de la Estadía
Comunicación y expresión (10%)	*Su volumen de voz es comprensible *Su expresión corporal manifiesta seguridad ejecutiva. *Su atuendo cumple los estándares de ser formal o semiformal. *Mantiene una línea de respeto en general.	*Su volumen de voz no es comprensible *Su expresión corporal manifiesta inseguridad. *Su atuendo cumple los estándares de ser formal o semiformal. *Mantiene una línea de respeto en general.	*Su volumen de voz no es comprensible *Su expresión corporal manifiesta inseguridad. *Su atuendo no cumple los estándares de ser formal o semiformal. *Realiza alguna falta de respeto.
Respuestas (10%)	Las respuestas son consistentes y con parámetros de lógica operativa.	Las respuestas no son consistentes pero tienen parámetros de lógica operativa.	Las respuestas son difusas e incoherentes.

Ponderación final de su exposición	40 /100
Datos de la exposición y evaluación de esta. 17 / Agosto / 2026	Manuel Benjamin Ortiz Martinez Bery Alm Profesor asesor institucional